NFC 配网的智能语音助手与私人物联网平台实现

Implementation of a Smart Voice Assistant with NFC-based Networking and a Personal Internet of Things Platform

NFC 配网的智能语音助手与私人物联网平台实现

摘　要

近场通信（NFC）技术，作为一种短距离无线通信标准，以其低功耗、快速配对及安全性高等特点，成为智能设备之间进行数据传输和网络配置的重要手段。然而，大多数市面上的语音终端接入公共物联网服务器，而非私人局域网，导致用户的个人隐私和数据安全可能面临诸多威胁。针对这一问题，本课题将结合 NFC 配网技术，开发智能语音助手终端，提出并构建一个既高效又安全的私人物联网平台。

本文从课题研究的背景和意义开始，简单介绍了 NFC 技术、Django 框架的原理与应用，叙述了关键词检测（KWS）技术的发展历程。然后，本文研究了语音处理、识别的理论原理；接着基于 Django 后端框架介绍了物联网平台的用户数据管理、设备绑定、大模型 API 接入、流媒体传输的设计方法；针对智能语音助手，基于 STM32 微控制器主控芯片提供了详细软硬件设计方案，实现了 NFC 与安卓手机通信并配网；为了在有限算力与资源的 MCU 上部署关键词识别系统，本文还提出了一种简单的关键词识别模型，使用卷积神经网络（CNN）模型对实时语音 MFCC（Mel Frequency Cepstral Coefficient）特征语谱图进行深度特征提取，再经过全连接层，然后对输出概率给予合适识别阈值，有效实现了唤醒词与噪音的二分类。最后用测试集分别在 Pytorch 与 STM32F446RET6 环境测试了模型推理的准确率，STM32F446RET6 模型推理准确率达到了 85.38%，相比 Pytorch 的 89.23%仅降低 4%，表现良好。

关键词：NFC，物联网平台，KWS，Django，STM32，MFCC，CNN

IMPLEMENTATION OF AN NFC-BASED NETWORKING SMART VOICE ASSISTANT AND A PERSONAL IOT PLATFORM

ABSTRACT

Near Field Communication (NFC) technology, as a short-range wireless communication standard, has become an essential method for data transmission and network configuration between smart devices due to its low power consumption, fast pairing, and high security. However, most voice terminals on the market connect to public IoT servers rather than private local networks, which poses potential threats to users' privacy and data security. To address this issue, this project integrates NFC-based network configuration to develop a smart voice assistant terminal and proposes the design and implementation of an efficient and secure private IoT platform.

This paper begins with an overview of the background and significance of the study, briefly introduces the principles and applications of NFC technology and the Django framework, and outlines the development of keyword spotting (KWS) technology. Then, it explores the theoretical foundations of speech processing and recognition. Based on the Django backend framework, this work describes the design methods of user data management, device binding, integration of large model APIs, and streaming transmission for the IoT platform. For the smart voice assistant, a detailed hardware and software design scheme based on the STM32 microcontroller is provided, enabling NFC-based communication and configuration with Android phones.

To support keyword recognition on a resource-constrained MCU, a lightweight keyword spotting model is proposed. This model uses a Convolutional Neural Network (CNN) to extract deep features from real-time MFCC (Mel Frequency Cepstral Coefficient) spectrograms, followed by a fully connected layer. A proper recognition threshold is then applied to the output probabilities, effectively achieving binary classification between wake words and noise. Finally, inference accuracy was evaluated on both the PyTorch environment and the STM32F446RET6 microcontroller. The model deployed on the STM32F446RET6 achieved an accuracy of 85.38%, only a 4% drop compared to the 89.23% achieved on PyTorch, demonstrating good performance.

Key words: NFC, IoT platform, KWS, Django, STM32, MFCC, CNN

目 录

1 绪论 1
1.1 引言 1
1.2 关键词检测技术研究现状 3
1.2.1 传统关键词检测技术 3
1.2.2 现代关键词检测技术 4
1.3 论文研究内容及章节安排 5
2 基于卷积神经网络的关键词识别系统 6
2.1 数据准备 6
2.2 Mel 频率倒谱系数特征提取 6
2.3 构建 CNN 关键词检测模型 10
2.4 本章小结 12
3 基于 Django 的私人物联网平台与安卓软件的实现 13
3.1 物联网平台前端基本用户交互界面设计 13
3.2 物联网平台 Django 后端服务器的实现 17
3.2.1 数据库与视图函数 17
3.2.2 网络的三种令牌机制：用户令牌、设备令牌、临时令牌 21
3.2.3 接入云计算 API 23
3.3 安卓应用设计 24
3.3.1 驱动类封装 24
3.3.2 用户登录注册 25
3.3.3 设备的 NFC 一碰配网与绑定 27
3.4 本章小结 29
4 NFC 配网的智能语音助手设计及关键词识别算法实践 30
4.1 硬件设计 30
4.1.1 语音助手主板电路设计 30
4.1.2 音频编解码器模块电路设计 35
4.1.3 元件焊接与装配 37
4.2 软件设计 38

4.3 本章小结 48
5 结论与展望 49
5.1 课题结论 49
5.2 未来展望 49
参考文献 50

1 绪论

1.1 引言

NFC 是一种短距离的高频无线通信技术，允许电子设备之间进行非接触式点对点数据传输。启发于非接触式射频识别(RFID)技术，NFC 结合无线互连技术实现了各信息设备的双向通信。大约在 2003 年的时候，索尼公司和当时的飞利浦半导体(现恩智浦 NXP 半导体)进行合作，计划基于非接触式射频卡技术研发一种更加安全快捷的并且能与之兼容的无线通讯技术。经过几个月的研发后，双方联合对外发布了一种兼容 ISO14443 非接触式卡协议的无线通讯技术，取名为 NFC(Near Field Communication)，具体通信规范称作 NFCIP-1 规范。

NFC 有三种工作模式：主动模式（亦称读卡器模式）、被动模式（亦称卡模拟模式）、双向模式（亦称点对点模式）。在主动模式下 NFC 终端可以作为一个读卡器，发出射频场去识别和读/写别的 NFC 设备信息；在被动模式下，此时 NFC 终端则被模拟成一张卡，它只在其他设备发出的射频场中被动响应，被读/写信息，在这种模式下，需要有一个 NFC 发起设备（如读卡器）工作在主动模式生成射频场，而另一个 NFC 目标设备（如智能手机）不必产生射频场，利用这个射频场进行负载调制(load modulation)，从而改变射频场将信息以相同的速度将数据传回发起设备；在点对点模式下 NFC 终端双方都主动发出射频场来建立点对点的通信。相当于两个 NFC 设备都处于主动模式。

虽然相比于蓝牙远距离无线通信，NFC 通信必须近距离甚至接触的建立连接方式，而且在传输速度上 NFC 最快只能达到 868Kbps，蓝牙却可以高达 2.1Mbps，但是 NFC 更加方便安全，而且成本相较于蓝牙更低，使其在隐私安全要求苛刻、指向性明确、及时响应、快捷操作的相关应用场景得以广泛应用[1]。目前 NFC 技术主要应用场景有：NFC 防伪溯源，使用 NFC 可以进行防伪查询验证，现在品牌酒已经在使用 NFC，包括奢饰品，服装等进行 NFC 防伪溯源；NFC 卡模拟交通卡，使用 NFC 刷公交地铁是目前来说被运用的最广泛的，受众群体也相对来说比较多，许多智能手机支持 NFC 功能来实现交通卡功能，免去了携带实体卡的麻烦；

使用 NFC 实现移动支付，主要是指带有 NFC 功能的手机虚拟成银行卡、一卡通等的应用，是带有 NFC 功能的手机可以作为一张银行卡在超市、商场的 POS 机上进行刷手机消费。2024 年支付宝推出的支付功能“碰一下”、2025 年美团采用恩智浦芯片在各大餐馆推广“左码右 NFC”都是比较新的应用赛道；使用 NFC 进行快捷配网，在两个都支持 NFC 功能的设备上，只要相互一碰，就可以实现网络配置信息传递，辅助建立 Wi-Fi、蓝牙连接。目前华为笔记本、手机的“Share 一碰传”及“多屏协同”功能就是典型的配网应用，这也是本文后续将采用的配网技术之一。

Django[2]，是用 python 语言写的开源 web 后端开发框架，并遵循 MVC 设计。劳伦斯出版集团为了开发以新闻内容为主的网站，而开发出来了这个框架，于 2005 年 7 月在 BSD 许可证下发布。这个名称来源于比利时的爵士音乐家 DjangoReinhardt，他是一个吉普赛人，主要以演奏吉它为主，还演奏过小提琴等。由于 Django 在近年来的迅速发展，应用越来越广泛，被著名 IT 开发杂志 SDTimes 评选为 2013SDTimes100，位列"API、库和框架"分类第 6 位，被认为是该领域的佼佼者。

Django 的主要目的是简便、快速的开发数据库驱动的网站。它强调代码复用，多个组件可以很方便的以"插件"形式服务于整个框架，Django 有许多功能强大的第三方插件，你甚至可以很方便的开发出自己的工具包。这使得 Django 具有很强的可扩展性。它还强调快速开发和 DRY(DoNotRepeatYourself)原则。框架采用了 MVC（Model-View-Controller）模式，其核心思想是分工、解耦，让不同的代码块之间降低耦合，增强代码的可扩展性和可移植性，实现了向后兼容，开发人员只需要将精力集中在业务逻辑的分析与实现。在 Django 中即 MVT（Model-View-Template）模式,其中 Model(模型)负责业务对象和数据库的映射；View(视图)负责处理 http 请求,返回响应给客户端；Template(模板)负责把页面展示给用户,处理数据到页面的渲染。本课题为了方便快速开发，构建私人物联网平台即使用了该框架作为后端。

关键词检测（Keyword Spotting, KWS）技术是语音识别领域的一个重要分支，旨在从连续语音流中识别出预定义的关键词或短语。随着智能设备的普及，关键词检测技术在智能家居、智能助手、智能穿戴设备等领域得到了广泛应用。

1.2 关键词检测技术研究现状

本小节将从传统方法到现代深度学习方法，详细回顾关键词检测技术的各阶段重要的发展历程。

1.2.1 传统关键词检测技术

传统关键词检测技术主要依赖于手工特征提取和经典模式识别算法。这些方法在早期计算资源有限的环境中发挥了重要作用，但随着数据量的增加和计算能力的提升，逐渐暴露出一些局限性。

（1） 动态时间规整

动态时间规整（Dynamic Time Warping，DTW）是一种用于测量两个时间序列之间相似度的算法，它通过计算关键词模板与输入语音之间的最佳匹配路径来确定关键词的存在。DTW 对时间尺度的变化具有一定的鲁棒性，适用于短语音信号的匹配，但计算复杂度较高，对长语音信号的处理效率较低，且难以处理背景噪声和语音变异性[3]。

（2） 隐马尔可夫模型

隐马尔可夫模型（Hidden Markov Model，HMM）是一种用于建模序列数据的统计模型，广泛应用于语音识别、自然语言处理和生物信息学等领域。它通过分析隐藏状态和观测数据之间的关系，来解决序列预测和分类问题。主要基于两个假设，马尔可夫假设：当前隐藏状态只依赖于前一个隐藏状态，即状态转换是无记忆的；观测独立性假设：当前观测数据只依赖于当前的隐藏状态，而与之前的观测数据无关。HMM 主要解决以下三个问题：

评估问题：给定一个模型和观测序列，计算该观测序列出现的概率。这可以通过前向算法或后向算法来解决。解码问题：给定一个模型和观测序列，找到最有可能产生该观测序列的隐藏状态序列。这可以通过维特比（Viterbi）算法来解决。学习问题：给定观测序列，调整模型参数以最大化观测序列的概率。这可以通过 Baum-Welch 算法（一种 EM 算法）来解决。

它作为一种理论基础扎实，算法成熟的模型，能够有效处理序列数据，捕捉数据中的时间依赖性和动态变化，但假设较为严格，实际数据可能不完全符合马尔可夫假设和观测独立性假设[4]，且需要大量的标注数据进行训练[5]。

（3） 基于样例的方法

基于样例的方法（Instance-Based Learning, IBL）是一种机器学习方法，它依赖于训练数据集中的特定实例或示例来做出预测或决策。在基于样例的学习中，学习过程主要涉及记忆训练实例，而不是将它们抽象成一个通用模型。当遇到新的查询实例时，算法会使用距离度量（例如欧几里得距离）将新实例与存储的实例进行比较。然后，算法使用最接近的实例来确定输出。样例学习算法中最著名的是 k-最近邻算法（k-NN）[6]，该算法根据 k 个最近邻的多数类对实例进行分类。基于样例的方法相对直观，容易理解和实现且由于基于样例的方法依赖于局部信息，对噪声和异常值具有一定的鲁棒性。但是其存储需求高且计算效率低，而且由于该方法主要依赖于训练数据，故对未见过的模式或数据分布变化的泛化能力较弱[7]。

1.2.2 现代关键词检测技术

随着深度学习技术的快速发展，关键词检测技术取得了显著的进步。深度习方法利用神经网络强大的特征提取能力，显著提高了关键词检测的准确率和效率。目前流行的现代关键检测方法就有如下几种：

（1）卷积神经网络

卷积神经网络（Convolutional Neural Network, CNN）是一种深度学习模型，最初用于图像处理，但近年来也被广泛应用于语音识别和关键词检测任务中。CNN 通过卷积层、激活函数、池化层和全连接层等结构，能够有效地提取语音信号中的特征，从而实现对关键词的检测。一种常见的方法是用 CNN 模型对梅尔倒谱系数（Mel Frequency Cepstral Coefficients）的频谱图进行深度特征提取，再通过全连接层进行分类[8]。为了适应嵌入式系统和移动设备，可以采用深度可分离卷积神经网络（Depthwise Separable CNN），以减少计算量和内存消耗，加快运算效率[9],但其推理准确率相比常规卷积有所降低。为了尽可能提高识别准确性，本课题将基于常规卷积方法提出一种卷积模型对唤醒词进行检测。

（2）循环神经网络及其变体

循环神经网络(Recurrent Neural Network, RNN)，以及其变体（如 LSTM 和 GRU）都是基于属于能够处理序列数据的神经网络架构，通过引入循环结构，能够记忆之前的信息，并将其用于当前的计算，从而捕捉序列中的动态变化。它特别适合处理具有时间依赖性的数据，如语音信号、文本序列和时间序列数据。

虽然其较传统的声学建模方法有很大优势，但相对较高的计算复杂度限制了这种神经网络的应用[9]。

(3) Transformer 架构

Transformer 是一种基于自注意力机制(Self-Attention)的深度学习架构，最初由谷歌的研究人员在 2017 年的论文《Attention Is All You Need》[11]中提出。主要由编码器和解码器组成。编码器负责处理输入序列并生成上下文表示，而解码器则利用这些上下文表示生成输出序列。每个编码器和解码器都由多个层组成，每层都包含多头注意力机制（Multi-Head Attention）和前馈神经网络（Feed-Forward Network, FFN）。其自注意力机制能够更有效地捕捉语音信号中的关键特征，相比传统的 CNN 或 RNN 模型，在处理长序列数据时表现更为出色，而且并行计算能力突破了序列模型的瓶颈，训练速度比循环神经网络快数倍。但也存在计算资源需求高，模型复杂性高，推理时延等问题[12]。

1.3 论文研究内容及章节安排

论文将针对私人物联网平台、智能语音助手进行设计，实现 NFC 与安卓手机通信并配网。其次，基于 CNN 模型设计了一个关键词检测模型，部署到语音助手上。论文主要内容和章节安排情况如下：

第 1 章：绪论。该章节主要简介了课题研究的背景，简单介绍了 NFC 技术、Django 框架的原理与应用，叙述了关键词检测（KWS）技术的发展历程。

第 2 章：基于卷积神经网络的关键词识别系统。该章节从语音预处理方法出发，研究梅尔频率倒谱系数提取过程，并提出并建立了一个基于卷积神经网络的语音频谱图特征的学习模型，用于特定语音唤醒词的检测。

第 3 章：基于 Django 的私人物联网平台与安卓软件的实现。该章节先阐述相关前端开发工作所运用的技术，研究了 Django 作为服务器后端框架的开发流程，介绍了物联网平台的用户数据管理、设备绑定、大模型 API 接入、流媒体传输这几种功能的设计方法。对移动端安卓 app 设计也提供了相关的设计思路。

第 4 章：NFC 配网的智能语音助手设计及关键词识别算法实践。该章节叙述了智能语音助手嵌入式开发设计的软硬件技术，并将本文提出的关键词检测系统部署到智能语音助手的嵌入式设备上，实现唤醒词与噪音的二分类，并测试验证了模型的性能。

2 基于卷积神经网络的关键词识别系统

2.1 数据准备

该关键词检测系统的唤醒词为“Hey Acho”，由于系统是一个分辨“关键词”和“非关键词”的二分类模型，故在进行模型训练之前，需要准备正负样本两种数据。正样本为室内环境下的“Hey Acho”音频片段，为了增强模型的泛化能力，采用数据增强的方法，使用 audiomentation 包对关键词进行音频变换（变速、变音高、加回声）来提供扩充样本；负样本包括室内背景噪音以及干扰词（“Hey Alice”或“Hello Alpha”等）。为了使音频格式规范化，音频的统一为 16KHz 采样率、单声道、采样位数 16bit、采样时间 1 秒。使用 Python 的 pyaudio 包进行音频样本的录制，每次录制的长度超出 1s 的部分裁剪，不足时补零确保长度相同，然后，给关键词样本标注为“1”，非关键词样本标注为“0”。准备好的各类型数据数量如下如表 2-1 所示。

表 2-1 各类型数据数量

数据类型	数量
“Hey Acho”	100
“Hey Acho”的扩充样本	200
室内背景噪音	250
干扰词	100

总共分为训练集与测试集两种数据集，各个数据类型平均抽取总数据的 80%作为训练集，其余的作为测试集。由于样本数量较少，不专门设验证集，而是使用 K 折交叉验证[13]从训练集中提取子集进行验证。

2.2 Mel 频率倒谱系数特征提取

语音关键词识别的关键一步是特征提取，这可以有效减少干扰信号并提高语音识别的准确性。通常，传感器获取的语音数据是在时域中的。在时域中，不容易区分不同声音的特征。因此，通过傅里叶变换和一定的处理来提取声音的特征。

MFCC（Mel 频率倒谱系数）是一种常见的声音特征提取方法[14]。MFCC 首先提取声音的频率域信息，由于人耳对声音的识别率并非线性，因此需要通过 Mel

滤波器组对获得的频率信息进行过滤，然后对结果进行离散余弦变换（DCT），以获得 MFCC 系数，这些系数就是声音的特征。本文主要通过以下五个步骤来获取音频的 MFCC 特征：

（1）对音频信号进行预加重和分帧处理。

（2）计算每帧的功率谱。

（3）使用 Mel 滤波器组进行滤波。

（4）进行离散余弦变换（DCT）。

将 DCT 变换后的 2~13 系数保留下来作为声音的特征。MFCC 特征提取过程如图 2.1 所示。

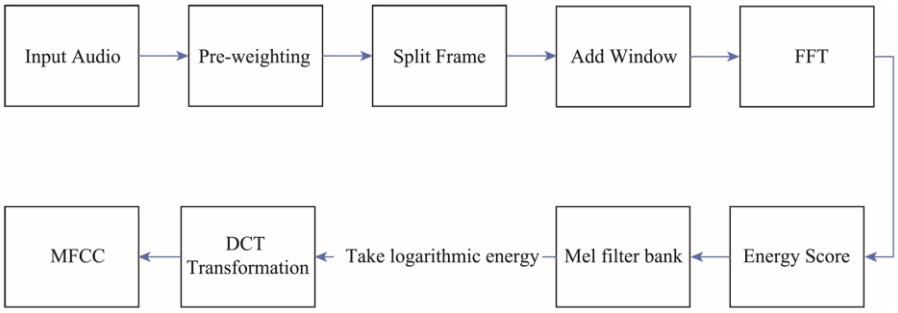


图 2.1 MFCC 特征提取流程图

首先，通过对原始音频信号进行预加重（Pre-weighting）来执行预处理，预加重滤波器在以下几个方面很有用：

(1)增强高频成分：语音信号的频谱特性通常是低频成分的能量较高，高频成分的能量较低。预加重通过增加高频成分的能量，使整个频谱更加平坦，有助于后续处理步骤（如傅里叶变换、倒谱分析等）的效果。

(2)提高信噪比：在实际应用中，高频成分往往更容易受到噪声的影响。预加重增强了这些高频分量，从而相对提高了它们的信噪比。

(3)减少信号失真：在一些信号处理和传输系统中，低频成分可能会引起过度的信号能量集中，导致失真。预加重可以平衡频谱，减少这种失真。

预加重处理其实是将语音信号通过一个高通滤波器，其计算公式如公式（2-1）所示：

$$Y(t) = x(t) - 0.97 \cdot x(t-1) \tag{2-1}$$

然后对预加重后的序列进行分帧、加窗。本文对采样来秒的音频信号大致划分为 99 个 25 毫秒的帧，为了确保信号的连续性，每一帧向后移动 10 毫秒，从

而使相邻帧之间存在重叠部分。随后，对每一帧的数据应用窗函数。窗函数通过在时间域中对信号进行加权处理，使每帧信号序列的两端平滑地衰减到零，从而减少信号在边界处的不连续性，其目的是消除各个帧两端可能会造成的信号不连续性（即谱泄露 spectral leakage）。常用的窗函数有方窗、汉明窗和汉宁窗等，常采用汉明窗（hamming window），其表达如公式（2-2）所示：

$$W(n)=(1-a)-a\cdot\cos(\frac{2\cdot\pi\cdot n}{M}) \qquad 1\le n\le M \tag{2-2}$$

其中 a 的取值不同，将会产生不同的汉明窗，一般情况下，$a=0.46$，M 为一帧中采样数量。为了方便，本文并未使用窗函数，而是保留了原始的分段信号。

接着，对于信号的每一帧，执行快速傅里叶变换（FFT）操作，如公式（2-3）所示，其中 N 表示离散傅里叶变换（DFT）的采样长度，若采样频率为 f_s，则 $X(k)$ 表示频率为 $k\frac{f_s}{N}$ 的能量向量。

$$X(k)=\frac{1}{N}\sum_{n=0}^{N-1}x(n)e^{-j\frac{2\pi}{N}kn} \quad 1\le k\le N \tag{2-3}$$

然后我们使用公式（2-4）计算功率谱（energy spectrum），对语音信号的频谱取模平方：

$$P(k)=\frac{1}{N}\left|X(k)\right|^2 \tag{2-4}$$

由于傅里叶变换的对称性，求得的功率谱图只需要前一半的频率，且包括直流部分（$k=0$），故仅截取前 $\frac{N}{2}+1$ 的频率部分，得到功率谱矩阵 $[P]_{C\times\frac{N}{2}}$，其中 C 表示总帧数。

紧接着计算 Mel 滤波器组，将功率谱通过一组 Mel 刻度的三角滤波器（通常为 26 个三角滤波器）来提取频带。采用这是因为对于相比于正常的频率机制，梅尔值更加接近于人耳的听觉机制，其特点是在低频范围内增长速度很快，但在高频范围内，梅尔值的增长速度很慢。每一个频率值都对应着一个梅尔值，其对应关系如公式（2-5）：

$$m=2595\cdot\log_{10}(1+f/700) \tag{2-5}$$

将梅尔频率 m 转换为频率 f，对上式（2-5）整理即可得到公式（2-6）：

$$f = 700 \cdot (10^{\frac{m}{2695}} - 1) \tag{2-6}$$

梅尔三角滤波器通常取26个，需要将正常频率最小值 $f_l = 0$ 和最大值 $f_h = \frac{f_s}{2}$ 用（2-5）式转换成对应梅尔频率的最小值 m_l 、最大值 m_h ，然后 $[m_1, m_h]$ 区间的频率分成 26 份，即总共需要 28 个梅尔分界频率，然后使用（2-6）将这些梅尔分界频率转换成正常频率，记为 $f(n),\ n = 0...27$ ，并用公式（2-7）求出各自的索引号：

$$g(n) = \frac{(1+N) \cdot f(n)}{f_s},\ n = 0...27 \tag{2-7}$$

接着利用公式（2-8）创建梅尔滤波器矩阵 H_i ，其中 i 为滤波器组序号：

$$H_i(k) = \begin{cases} 0 & ,k < g(i-1) \\ \dfrac{k - g(i-1)}{g(i) - g(i-1)} & ,g(i-1) \le k \le g(i) \\ \dfrac{g(i+1) - k}{g(i+1) - g(i)} & ,g(i) \le k \le g(i+1) \\ 0 & ,k > g(m+1) \end{cases} \quad k = 0...\frac{N}{2}, i = 0...25 \tag{2-8}$$

信号通过 Mel 滤波器组后，对 26 个能量数据取对数得到梅尔能量矩阵 $[M]_{C\times 26}$ 即公式（2-9），其中，C 为总帧数，P 为功率谱矩阵，H 为梅尔滤波器矩阵：

$$[M]_{C\times 26} = \log([P]_{C\times\frac{N}{2}}[H^T]_{\frac{N}{2}\times 26}) \tag{2-9}$$

由于梅尔能量矩阵组每帧的 26 个系数是相关性很高的，因此我们用离散余弦变换（DCT）来去相关并且降维。离散余弦变换公式如（2-10）所示：

$$F(i,n) = \sum_{m=0}^{25} M(i,m) \cdot \cos[\frac{\pi \cdot n \cdot (2m+1)}{2 \times 26}],\qquad n = 0...25 \tag{2-10}$$

最后对离散变换后的梅尔能量矩阵 $[F]_{C\times 26}$ 进行升倒谱计算，从而获得音频的 MFCC 特征，一般来说，在自动语音识别领域，因为大部分信号数据一般集中在变换后的低频区，所以对每一帧只取前 13 个数据作为该帧的 MFCC 特征，升倒谱公式如式（2-11）：

$$J(i) = 1 + (\frac{L}{2}) \cdot \sin(\frac{\pi \cdot i}{L}),\qquad i = 1,2,3...,13 \tag{2-11}$$

得到升倒谱矩阵 $[K]_{13\times 1}$ ，取离散变换后的梅尔能量矩阵 F 的前 13 个特征（即

$[F]_{C\times13}$）进行升倒谱最终得到 MFCC 特征图，如式（2-12）：

$$[MFCC]_{C\times13}=[K]_{13\times1}[F]_{C\times13} \tag{2-12}$$

对每各数据进行 MFCC 特征提取并将每一帧合并在一起得到各自的 MFCC 特征语谱图。图 2-2 显示了不同数据类型的 MFCC 特征语谱图。

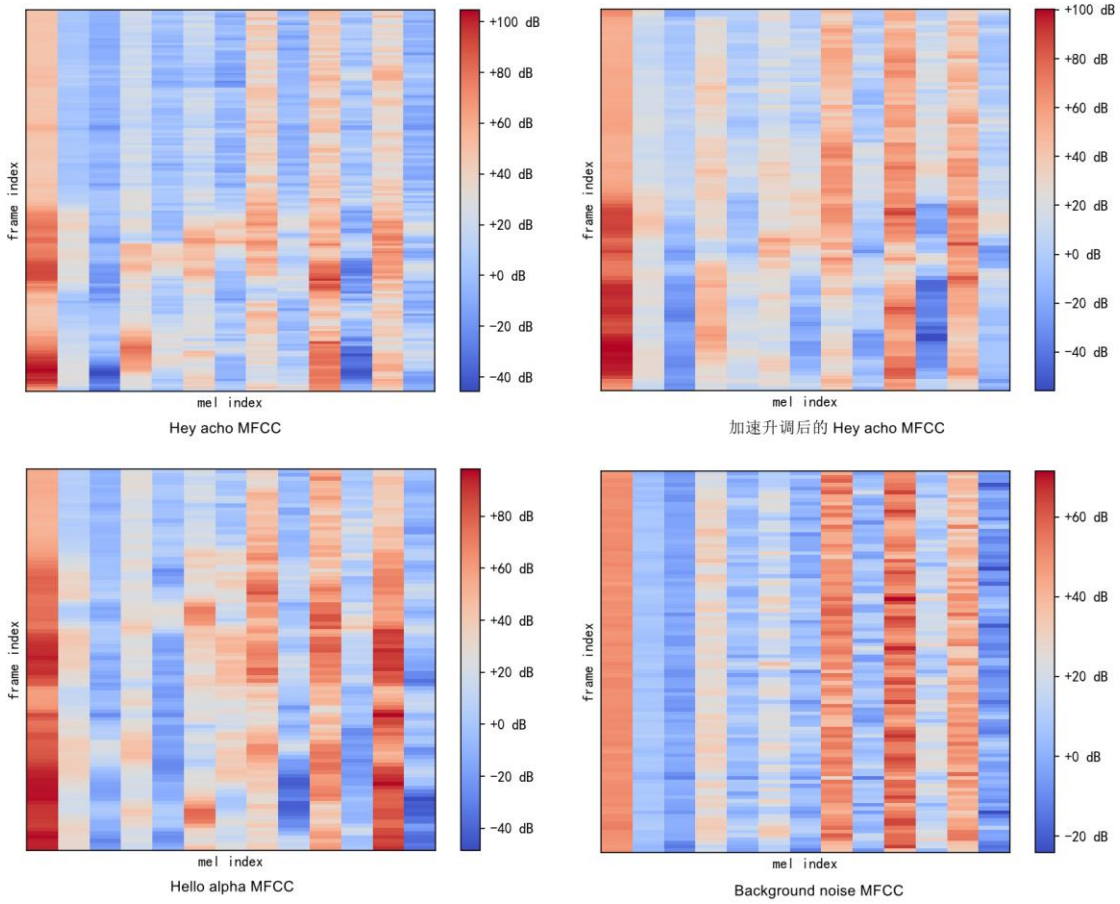


图 2.2 不同数据类型的 MFCC 特征语谱图

2.3 构建 CNN 关键词检测模型

卷积神经网络（CNN）在图像识别领域有着广泛的应用。CNN 中的卷积核能够以较少的参数从图像中提取更深层次的特征，因此在图像分类任务中表现出色。基于上述操作，将音频信号转换为图像信息后，语音识别和分类问题可以转化为与传统图像识别相同类型的问题。

在本文中，使用 CNN 对语音的 MFCC 特征图进行分类和识别。该网络旨在在微控制器上实现，并使用 Pytorch 构建。如图（2-3）卷积神经网络模型结构图所示，这是一种用于唤醒词检测的轻量级卷积神经网络模型，输入为音频信号的

MFCC 特征图，通过多层卷积和全连接层实现对语音中是否包含指定唤醒词的判断。模型首先使用一个包含 4 个 3×3 卷积核的卷积层提取基础声学特征，随后通过 ReLU 激活函数，并在该卷积层输出后添加一个 2×2 的最大池化层，用于将特征图的尺寸缩小一半，从而有效降低计算复杂度。接着是第二个卷积层，使用 8 个 3×3 卷积核进一步提取高阶特征，同样经过 ReLU 激活函数，并在其输出后添加最大池化操作，继续将特征图的空间尺寸减半。卷积层输出后被展平为一维向量，输入到全连接网络部分。第一个全连接层包含 64 个节点，用于学习更复杂的特征组合关系；随后是第二个包含 32 个节点的全连接层进一步压缩并聚焦特征，两个全连接层之间均使用 ReLU 激活函数。最后一层为输出层，仅包含一个节点，使用 Sigmoid 激活函数输出一个 0 到 1 之间的概率值，用于表示输入音频为唤醒词的置信度。在训练过程中，模型使用二元交叉熵损失函数（BCELoss）进行优化，标签为 0 或 1，分别表示“非唤醒词”和“唤醒词”。推理时，设定一个阈值（如 0.5）对输出结果进行二值化判断。该模型结构简洁高效，适用于在资源受限的设备（如微控制器）上进行实时唤醒词识别任务。二元交叉熵总损失函数公式为（2-13）：

$$\varepsilon = -\frac{1}{X}\sum_{i=1}^{X}[y_i \cdot \log(p_i) + (1-y_i)\cdot \log(1-p_i)] \qquad (2\text{-}13)$$

其中 X 表示用于训练的总样本数；y_i 是关于第 i 个输入数据的符号函数，当第 i 个输入数据为“唤醒词”时，y_i 取 1，若为“非唤醒词”时，y_i 取 0；p_i 为网络对于第 i 个数据属于“唤醒词”的输出概率。

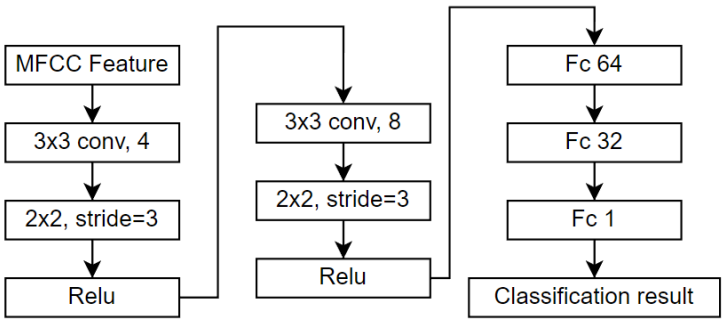


图 2-3 卷积神经网络模型结构图

在训练过程中，数据集通过 Pytorch 内置函数进行洗牌，以避免网络学习到顺序信息，并提高模型的最终准确性。Adam 优化器用于梯度更新，具有实现简

单和计算效率高的特点。它可以自动调整学习率并控制步长，有助于模型快速准确地找到最优参数。在上述参数设置下，模型训练了 100 个 epochs，训练结果如图 2-4 和图 2-5 所示。

从图中的训练结果可以看出，经过多次迭代后，模型已经达到了较高的准确率。训练集的准确率接近 100%，训练集损失接近 0。验证集的准确率最高达到了 90.11%，表明该模型具有良好的分类性能。

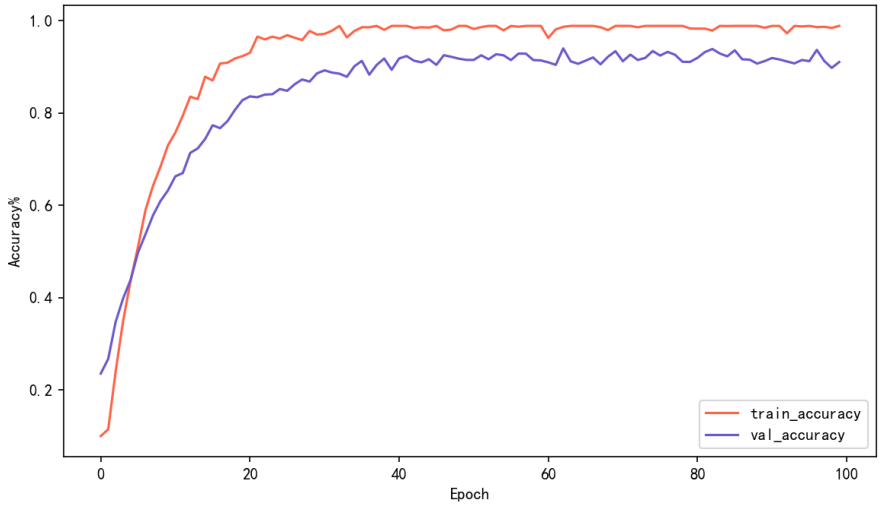


图 2-4 检测词模型训练过程中准确率示意图

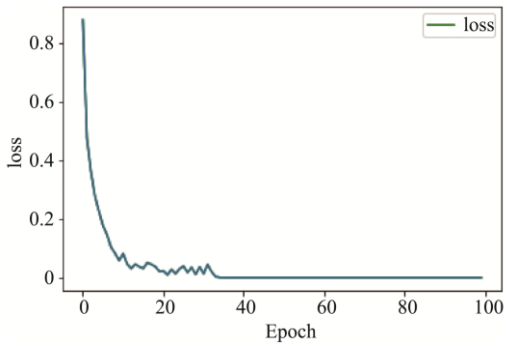


图 2-5 模型训练过程中损失示意图

2.4 本章小结

本章首先对梅尔频率倒谱系数特征提取方法的理论基础进行了介绍，讲解了 MFCC 从采样数据的预处理、FFT 变换到三角余弦变换的详细运算过程，并分析了各种处理方式的原因，最后简单说明了本文使用的 CNN 关键词检测模型的框架与训练过程和结果。

3 基于 Django 的私人物联网平台与安卓软件的实现

物联网平台前端基本采用“CSS”、“JavaScript”、“Html5”的前端三件套来实现，后端采用当前较为流行的 Django 框架，安卓配套的 APP 使用 Android Studio IDE 来开发。

该平台具有用户注册与登录的功能，以及具体网页的操作（如更改用户头像、查看用户基本信息、设备选择与控制操作等）。安卓应用使用 Android Studio 进行开发，该 APP 具备登录注册功能，以及设备扫码/NFC 感应两种方式实现绑定配网、拍照、图像处理、网络文件传输的功能。整个私人物联网系统的大体框架如图 3-1 所示：

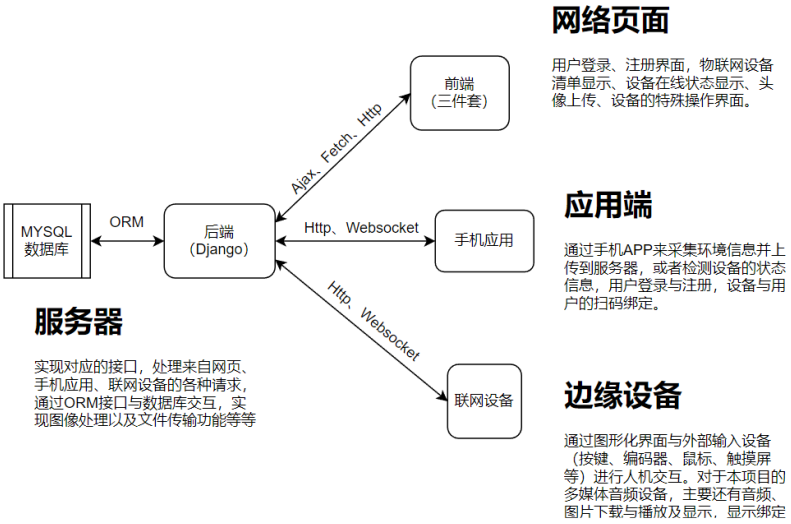


图 3-1　私人物联网系统框架

3.1 物联网平台前端基本用户交互界面设计

主要使用 Html、JavaScript、CSS 基础前端开发三件套实现，个别页面为了开发方便，使用了 Bootstrap5 框架。同时引入了 jQuery 组件简化前端 JS 开发。

(1)用户登录与注册界面

注册界面有用户名、密码输入、重新输入三个输入框，当用户名输入框的焦点失去时，该页面会向后端发送 Ajax 请求验证用户名的存在情况，并加以提示。点击注册，会对两次输入密码进行验证，并由服务器验证通过后才成功注册，若不一致也会有标红提示信息。图 3-2 是页面在不同状态下的情况：



图 3-2　注册页面的不同状态示意图

登录界面和注册页面差不多，主要区别是登录界面具有密码提交与验证正误功能如图 3-3 所示：



图 3-3　登录页面的不同状态示意图

（2）物联网平台主页面

当用户登录信息有效时，界面即跳转到物联网平台主页面“Home”，主页面点击顶部导航栏“MyDevices”按钮即可显示用户所拥有（已经绑定）的所有设备卡片组成列表，如图 3-4 所示主面板中以 Flexbox 流式布局的方式显示了多个已绑定的可用设备卡片，其中仅显示了有一个名为“Acho”的设备，该设备即本课题设计的智能语音助手，显示了其唯一 MAC 地址以及联网状态：

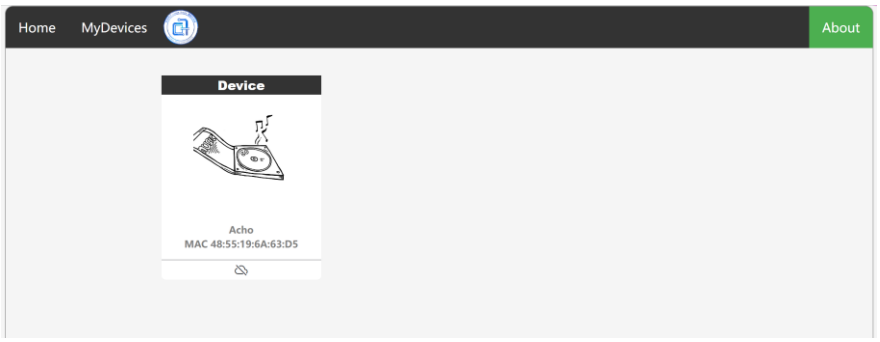


图 3-4　物联网平台 Home 面板的设备清单界面

点击用户头像，即可显示用户的有关信息，如图 3-5 所示，信息面板显示了用户名、注册时间、以及拥有设备的信息（之后可以加入其他更丰富的信息）。



图 3-5　用户简介界面

点击头像，即可显示模态框，可以拖入新的图像来更换用户头像（图 3-6），亦可以点击模态框中的“恢复默认”按钮来恢复默认的图像（图 3-7）。



图 3-6　上传新用户头像的多模态框



图 3-7　用户头像恢复默认的示意图

(3)设备应用界面

以“智能语音助手”设备为例，在 HOME 界面下点击对应的设备卡片可以进入相应设备的应用功能界面。图为设备“Acho”的应用界面，界面分为两栏，左边栏为下拉功能菜单栏，分别有图片、音频、更改头像、我的主人（显示设备所有拥有者信息）。

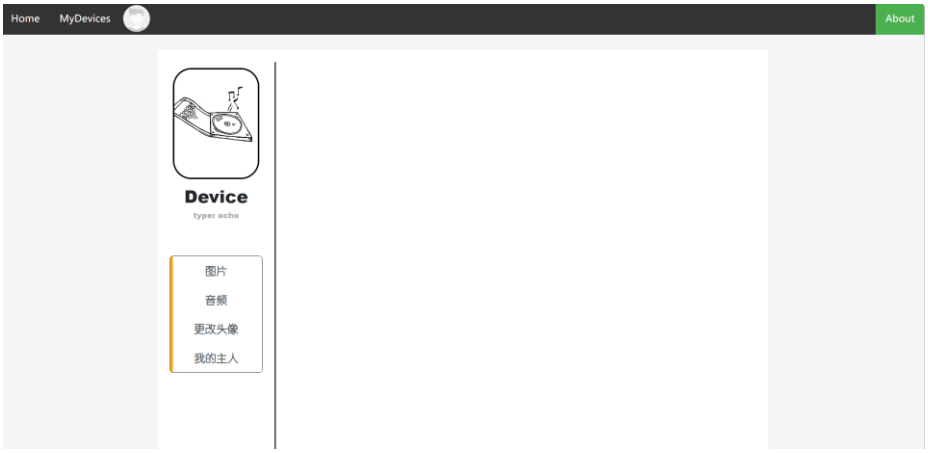


图 3-8　用户头像恢复默认的示意图

点击“图片”按钮，即可显示设备所拥有的图片列表（图 3-9）：

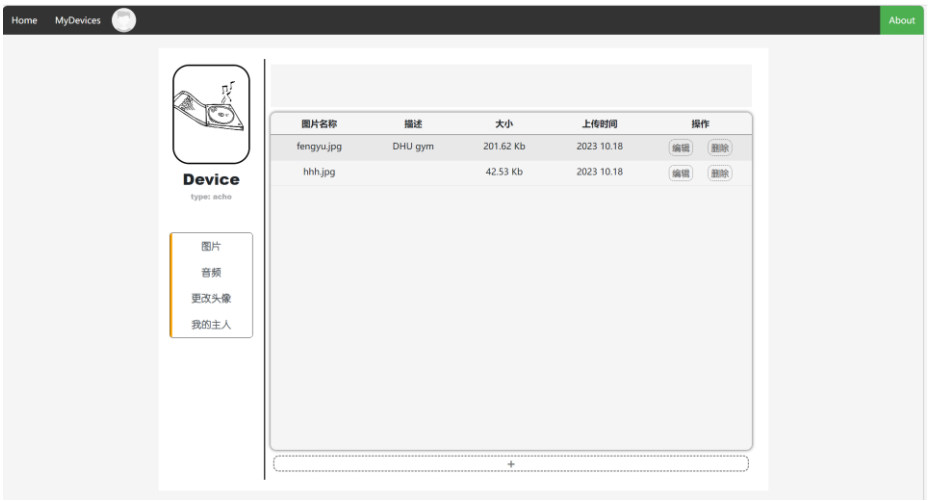


图 3-9　设备的图片列表

点击列表页面最底端的“+”长型按钮，同时亦可以点击每条子项目右面操作栏中的“编辑”或“删除”按钮更改子项目以及删除图片，将图片上传给后端，文件与该设备在数据库中建立一对多的关系，设备也可以联网下载该资源，如图 3-10 所示，“音频”按钮与“图片”按钮功能类似，不额外展开说明。

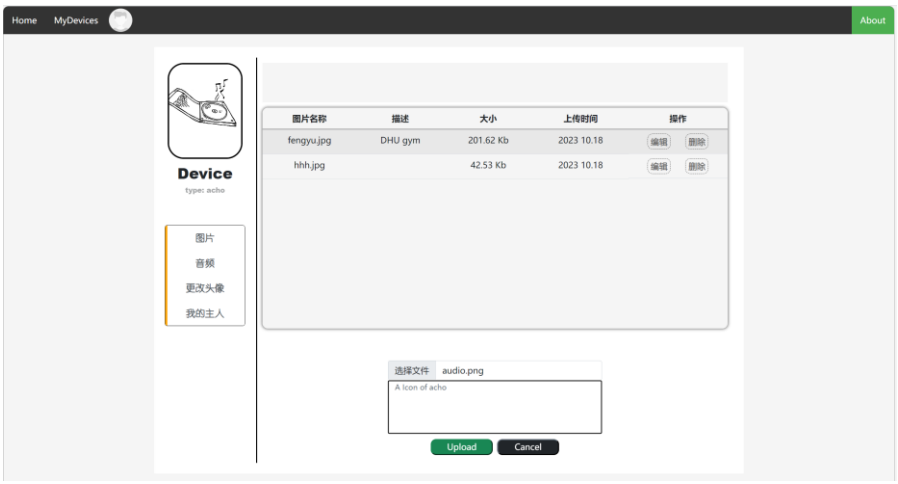


图 3-10　设备的图片列表

3.2 物联网平台 Django 后端服务器的实现

本节将利用 Django 中 MVT（Model-View-Template）模式在数据库（Model）、视图函数(View)、前端模板（Template）进行设计。数据库采用 MySQL 维护物联网系统用户与设备的信息。采用 WebSocket 协议以支持服务器与边缘设备流媒体（音频流）的快速传输，为了增强访问隐私性与数据传输安全性，采用 Flask 框架的 itsdangerous 组件构建了物联网系统的网络令牌机制，以对数据进行哈希加密处理，并实现 TTS（Text To Speech）、STT（Speech To Text）、大语言模型 API 接口。

3.2.1 数据库与视图函数

建立三个基本的应用（图 3-11），分别为“appWelcome”、“appHome”、“appMyBroker”。以及一个设备的应用，即“appAcho”，若后续加入多种设备，每个设备都由单独的应用来实现其功能。

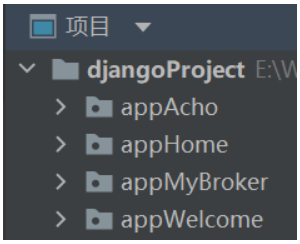


图 3-11　后端项目中的四个应用

（1）appWelcome

该应用实现的是基本的平台用户信息以及设备信息的收集。

首先是 models（数据库模型建立），该应用主要包含了用户（User）、设备（Device）、用户权限（UserAuthority），以及设备类型（DeviceType）总共四张表，他们之间的关系如下面框图 3-12 所示：

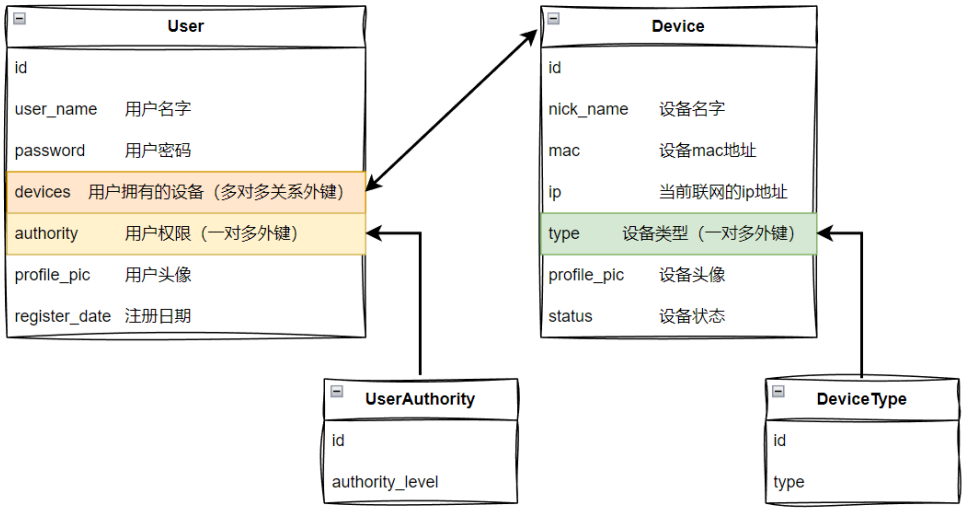


图 3-12　appWelcome 数据库模型关系图

其次是视图函数的实现，分别有“passwordCheck”（判断登录密码，若正确则返回网络密匙“token”，错误则返回错误信息）、“userCheck”（判断用户名是否存在）、“login”（响应登录页面）、“register”（响应注册页面）。由于篇幅限制，主要展示“passwordCheck”的实现如下图 3-13 所示：

```
def passwordCheck(request):
    if request.method == 'GET':
        user_input_name = request.GET.get('username')
        user_input_password = request.GET.get('password')
        try_user = models.User.objects.filter(user_name=user_input_name).first()
        res = {"status": "invalid-user"}
        if try_user:
            if user_input_password == try_user.password:
                user_id = try_user.id
                info = {
                    'user_id': user_id,
                    'user_name': user_input_name,
                }
                token = UserTokenGenerator.dumps(info).decode('utf-8')
                try_user.is_active = True
                try_user.save()
                res["status"] = "corrected"
                res["token"] = token
            else:
                res["status"] = "uncorrected"
        return JsonResponse(res)
```

图 3-13　passwordCheck 视图函数源码

（2）appHome

该应用实现的接口主要是主页面的设备信息请求、用户信息的获取与校验、更换用户头像等。

分别对用户、设备实现了视图函数，用户访问的视图函数如表 3-1 所示，设备访问的视图函数如表 3-2 所示：

表 3-1　appHome 应用的用户访问视图函数

视图函数名	功能
userHome	响应主页面
userProfile	响应用户详情页
devicePanels	响应设备列表页
userGetDeviceData	获取用户所有设备信息
userProfileUploadAvatar	用户上传图像
userAvatarSetDefault	恢复用户默认头像
showErrorPage	返回错误页面
userBind	用户-设备绑定接口
userDeviceSetNickName	修改设备昵称

表 3-2　appHome 应用的设备访问视图函数

视图函数名	功能
deviceInit	设备初始化，通过该接口获取到设备令牌
deviceGetBindQRCode	通过该接口获取用于设备的绑定二维码
deviceGetBindNFCCode	通过该接口获取用于设备的绑定 NFC 哈希字符串

appHome 应用中设备访问接口为设备通用的接口函数，针对不同的设备，若要实现其他特异的接口应该在各自的应用中实现，如智能语音助手的特异接口在“appAcho”中实现。

（3）appAcho

该应用实现的是智能语音助手（Acho）的功能，其数据库模型如框图 3-14 所示，用户访问的视图函数如表 3-3 所示，设备访问的视图函数如表 3-4 所示：

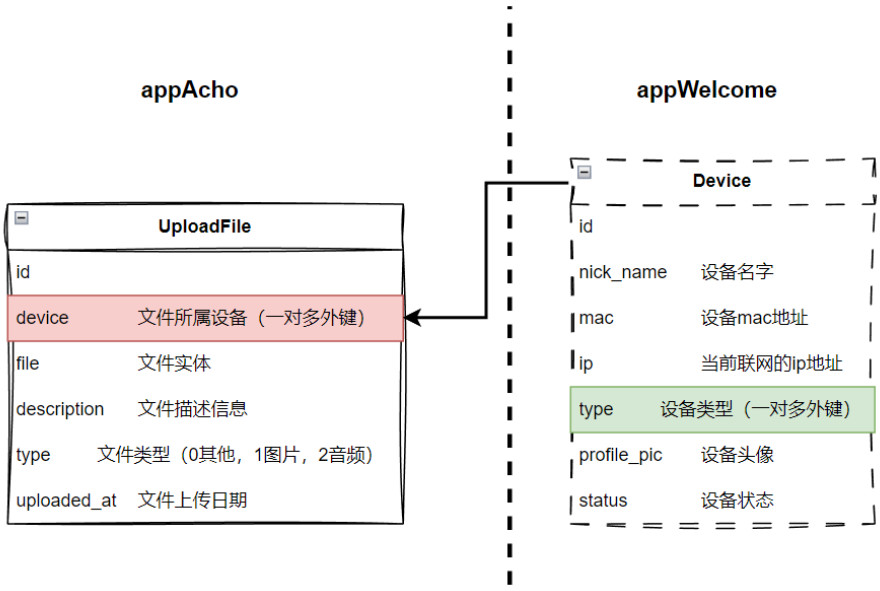


图 3-14　appWelcome 数据库模型关系图

表 3-3　appAcho 应用的用户访问视图函数

视图函数名	功能
homeImage	响应图片列表页面
homeAudio	响应音频列表页面
avatarSet	设置设备头像
avatarSetDefault	恢复设备默认头像
myOwners	响应持有者列表
uploadFile	上传文件接口
deleteFile	删除指定文件
getDeviceFileList	返回设备的所有特定类型的文件信息

表 3-4　appAcho 应用的设备访问视图函数

视图函数名	功能
deviceGetFileList	获取特定类型的文件信息
deviceGetAudioFile	分块获取指定的音频数据流
deviceGetDitherBin	获取指定抖动处理并压缩后的图片二进制文件

其中“deviceGetDitherBin”（获取指定抖动处理并压缩后的图片二进制文件）使用灰度抖动算法对灰度图进行拟灰度处理，这是因为智能语音助手使用的是单色屏（即黑白屏）。区别于简单的对图像进行二值化处理，图像灰度抖动算法可以将任何 RGB 或灰度图转换成仅由黑白像素点构成的“拟灰度”的图像，即通过控制黑白像素点的分布密度来实现不同灰度的效果。适用于各种单色屏的灰度图像显示，使得图像更加有层次，尽可能保留了原本图像的纹理。算法的实现源码如图 3-15 所示，采用 OpenCV 库对图像进行处理：

```
Mat getImgDithered(const string& address){
    Mat img_read = imread(address,IMREAD_COLOR);
    Mat img;
    vector<Mat> img_chanels;
    split(img_read,img_chanels);//分离BRG三通道
    img= 0.299*img_chanels[0]+0.587*img_chanels[1]+0.114*img_chanels[2];//组合成灰度图
    float err = 0;
    img.convertTo(img,CV_32F);//为了避免误差传递过程的溢出，先转换为32F类型
    for(int32_t r=0;r<img.rows;r++)
    {
        for(int32_t c=0;c<img.cols;c++)
        {
            if(img.at<int>(r,c)>128)
            {
                err = (float)img.at<float>(r,c) -255;//计算正向误差
                img.at<float>(r,c)=255;//像素点设置为白色
            } else{
                err = (float)img.at<float>(r,c);//计算反向误差
                img.at<float>(r,c)=0;//像素点设置为黑色
            }
            if(r+1<img.rows&&c+1<img.cols&&c-1>=0)//以下为误差补偿操作
            {
                img.at<float>(r,c+1)+=err*7/16;
                img.at<float>(r+1,c)+=err*5/16;
                img.at<float>(r+1,c+1)+=err*1/16;
                img.at<float>(r+1,c-1)+=err*3/16;
            }
        }
    }
    img.convertTo(img,CV_8U);//转化成8U的类型
    return img
}
```

图 3-15　灰度抖动算法实现的源码

3.2.2 网络的三种令牌机制：用户令牌、设备令牌、临时令牌

Flask 的 itsdangerous 模块是用于处理令牌（tokens）和加密、签名等安全操作的工具。这个模块非常适合在 Flask 应用中实现令牌机制，比如生成安全的身份验证令牌、URL 令牌、重置密码令牌等。最为重要的是，该组件生成的密文同时具备时间校验以及完全信息的描述，作为令牌只需要解析客户端上传的密文即可获得全部信息，验证过程不占用任何服务器本地内存，且安全可靠自带安全加密。其操作简单，且兼顾了安全性和一定的性能，对比 HTTP Cookie 机制与 Redis 生成令牌机制取其长补其短，非常实用。本项目的所有令牌机制均采用此机制运作。

（1）用户令牌

针对用户客户端（浏览器、手机应用）生成身份令牌，用于每一次用户在客户端进行敏感操作（更改头像、绑定设备、获取设备信息、操作具体设备等）时的身份验证，对于浏览器，每次登录成功后服务器会派发该令牌，或者每一次敏感操作后更新令牌。服务器验证令牌失败则会返回错误信息，前端重定向至错误界面（图 3-16）。



图 3-16　用户令牌验证失败前端重定向的错误界面

（2）设备令牌

平台中的设备访问接口同样也需要设备上传对应的设备令牌，设备每一次上电且联网后上传自己的信息向服务器请求初始化设备，验证通过即可获取该令牌。设备每一次进行敏感请求操作都必须将该令牌附上，否则将遭到服务端的拒绝访问（Access Forbidden），同样也具备令牌更新机制，以确保身份安全性。

（3）临时令牌

该令牌针对一些特别敏感的场合（设备绑定密文验证操作、密码修改操作等），通常过期维持时间特别短，采用特定的哈希加密，保障安全信息不被恶意截取或篡改。设备的绑定二维码解析的信息即为其中一种临时令牌。

服务器将各种令牌的验证过程函数封装成了闭包结构的装饰器。视图函数若需要进行相关身份验证只需要在函数头附上对应装饰器的语法糖即可。图(3-17）和图（3-18）分别为用户令牌、设备令牌装饰器修饰示例：

```
@user_token_check
def homeImage(request, tok, user):
    if request.method == "GET":
```

图 3-17　用户令牌装饰器修饰

```
@device_token_check
def deviceGetFileList(request, json_data, device):
    if request.method == 'GET':
```

图 3-18　设备令牌装饰器修饰

3.2.3 接入云计算 API

一般的边缘嵌入式设备算力与资源有限，无法部署各种较大的人工神经网络模型，故需要借助市面上一些提供云计算的厂商的 API 接口来实现智能的功能，本课题的智能语音助手需要实现 TTS（Text To Speech）、STT（Speech To Text）、大语言模型推理这三种应用。国内外目前有各大云计算厂商，提供 TTS、STT 应用的厂商有美国谷歌公司的 Google Cloud Platform、亚马逊公司的 Amazon Web Services，国内比较有名的如腾讯云(Tencent Cloud)，华为云 (Huawei Cloud)，科大讯飞开放平台等。大语言模型厂商有美国 OpenAI 公司 ChatGPT、xAI 公司的 Grok，国内的深度求索公司的 DeepSeek、月之暗面公司的 Kimi 等。

本课题主要采用了腾讯云的“语音识别 API”、“语音合成 API”，以及 deepseek 的 DeepSeek-V3 API。整个调用与信息传输流程如图 3-19 所示，箭头线上的数字代表信息传输的顺序。首先语音助手嵌入式板卡将采集的音频文件通过 websocket 协议发送给物联网后端，然后后端将通过 http 协议先将音频文件传输给百度 STT API，再将得到语音识别文字结果发送给 DeepSeek-V3 API，得到大语言模型回答的文本信息，接着将回答信息发送给百度 TTS API 请求对应得语音生成音频数据，最后，后端将语音生成得结果回传给语音助手，由语音助手播放音频。

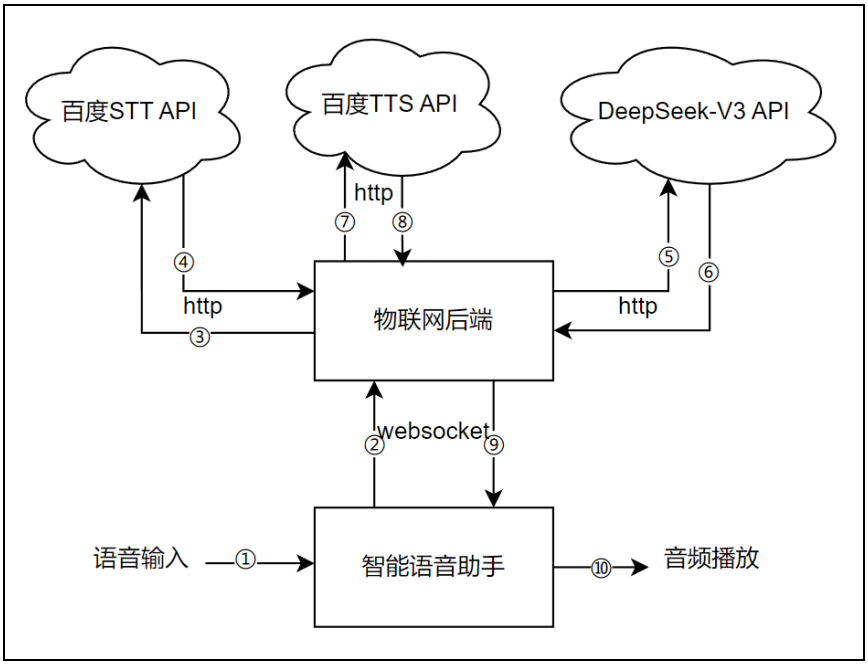


图 3-19　API 调用与信息传输流程图

其中 WebSocket 通道建立采用了 channels 包，为了能够进行分组会话，实现“设备-设备”,“用户-设备”的共同会话通道，构建了基于主题的订阅和广播机制。

在本项目中其主要的用途：

1. 用户与设备纯文本的传输、大文件（音频图片数据）的传输。

2. 检测设备状态，通过间隔的心跳包来尝试获取设备的 pong 响应。

3. 服务器后端主动发送控制信息，如唤醒设备等，实现双向的信息交互功能。

3.3 安卓应用设计

3.3.1 驱动类封装

移动端采用 AndroidStudio 开发配套应用，使用 Java 与 C++语言混合开发，其中运用了安卓内置 NDK 组件对 C++代码进行交叉编译。

为了体现代码结构以及程序架构的清晰，预先对安卓系统的“相册”、“相机”、“HTTP”等应用模块的驱动进行了类的封装，使得原本复杂的系统权限申请、URL 解析等步骤被隐藏了起来，这也方便了实际应用的开发。图 3-20 所示为封装的相关类：

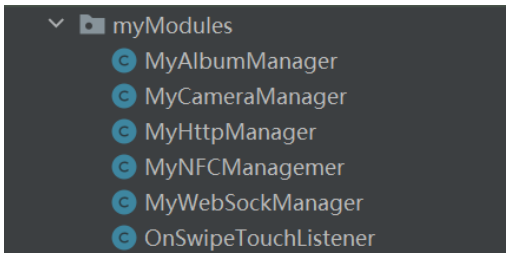


图 3-20　应用模块封装类

相册模块 MyAlbumManager、相机模块 MyCameraManager 使用系统内置的多媒体内容提供器“android.provider.MediaStore”来获取 url 资源。HTTP 模块 MyHttpManager 以及 Websocket 模块 MyWebSockManager 使了用 okhttp3 包。为了实现二维码扫描功能，使用了开源包“com.yzq.zxinglibrary”调用摄像头扫描二维码并由活动返回解析结果。智能语音助手的 NFC 芯片为意法半导体的 st25dv16k，且官方提供了 ST25 NFC Tap App，可用于读取和写入 ST25D 标签的案例，在例程上提取关键驱动部分封装成 MyNFCManager 模块。

3.3.2 用户登录注册

每一个 app 用户需要注册一个账号并保持登录状态才可正常使用应用的各项功能。打开应用首先进入登录/注册界面，如图 3-21a 与图 3-21b 所示，按下 Sign up 或 Sign in 按键，可以动态切换到注册或登录界面，同时具备基本的用户名存在提示以及密码纠错功能；登录成功后，界面跳转到应用功能页面（图 3-22），分别有五个按键，对应不同功能，从上至下依次为“切换账号”、“通过 NFC 配置设备网络”、“通过 NFC 方式绑定设备”、“通过二维码访问绑定用户”以及“发送”（用于直接发送 websocket 信息转发给指定设备）

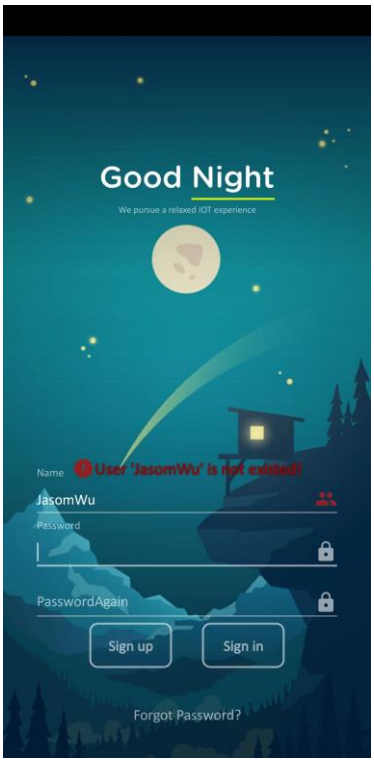


图 3-21a 用户注册界面



图 3-21b 用户登录界面

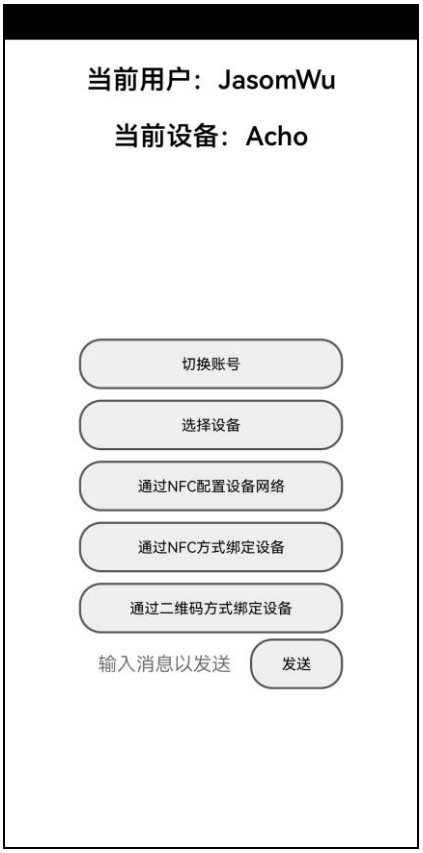


图 3-22　应用功能界面

3.3.3 设备的 NFC 一碰配网与绑定

一般而言，具备联网功能嵌入式设备都有连接网络的步骤，私人物联网系统部署在室内，在家庭和小型企业网络中，路由器通常被配置为网关。边缘设备通过 Wi-Fi 接入网关，设备需要从外部得知 Wi-Fi 的 SSID 和密码。而当设备不具备人机文本交互硬件条件时，就可以通过蓝牙或 Smart-Config（手机 APP 通过广播或组播报文将加密的 Wi-Fi 信息发送给设备）等方式辅助配网。然而蓝牙和 Smart-Config 配网方式存在一定的网络安全和成本问题：

在蓝牙配对过程中，如果使用“Just Works”模式（即无需用户输入任何信息即可完成配对），攻击者可以拦截并篡改蓝牙设备之间的通信数据，从而获取敏感信息或控制设备，设备支持蓝牙通信也有一定的硬件成本。

Smart-Config 通过无线信号将 Wi-Fi 密码传输给设备，攻击者可以在信号覆盖范围内监听这些数据，从而获取 Wi-Fi 密码；即使使用了加密算法，如果加密密钥被逆向工程破解，攻击者仍然可以解密获取 Wi-Fi 凭据[15]；Smart-Config 需要路由器的参与，不同品牌的路由器可能存在兼容性问题，这可能影响配网成功率。

本节提出了一种新的配网方式——NFC 一碰配网，NFC 成本低廉，无需过多外部条件支持，且利用 NFC 近场通信的优势，从而避免敏感配网信息暴露在无线电波环境中，这些特性将使配网过程更加安全、便捷、低成本。

（1）NFC 一碰配网：

登录 APP 账户后，点击应用功能界面的“通过 NFC 配置设备网络”，接着跳转到“配置设备网络信息”界面（图 3-23），在两个输入框中分别输入 Wi-Fi 名称与密码后点击“开启 NFC 配网按钮”，跳转到 NFC 配网提示界面（图 3-24），将手机靠近设备的 NFC 感应区，当弹出提示消息“设备已完成配网！”即完成了配网。



图 3-23　配置设备网络信息界面



图 3-24 NFC 配网提示界面

（2）NFC 一碰绑定：

设备与用户之间需要建立从属关系，登录后点击应用功能界面的“通过 NFC 方式绑定设备”，接着靠近设备 NFC 感应区等到弹出“成功绑定设备！”，即表示完成了绑定操作（图 3-25）。然后，点击应用功能界面的“通过 NFC 方式绑定设备”，可以看到绑定的设备已添加到当前用户的设备列表中（图 3-26）。



图 3-25　NFC 设备绑定提示界面

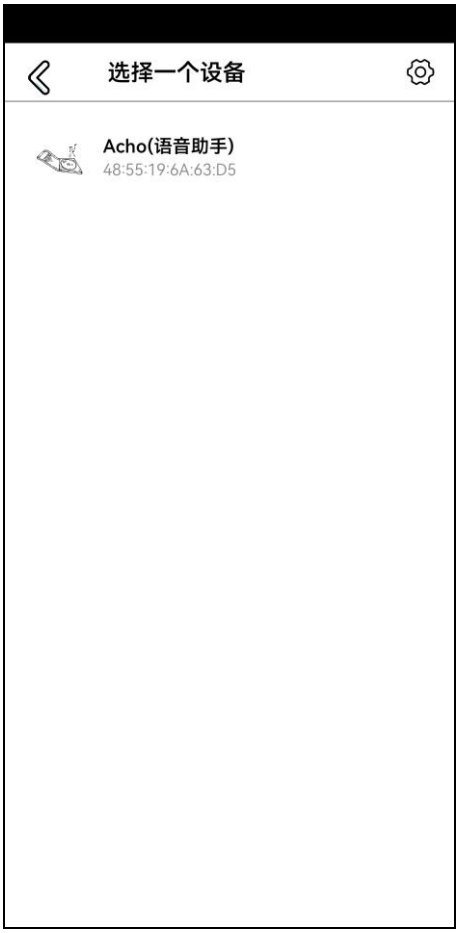


图 3-26　设备选择列表界面

3.4 本章小结

本章首先对私人物联网系统运作的总体框架进行了介绍，然后对物联网平台前端页面的设计方法与技术进行了说明，接着基于 Django 框架对后端服务器的数据库构建、视图函数逻辑进行了介绍，提出了“物联网的三令牌机制”解决了数据与访问权限安全维护问题，接着阐述了云计算 API 的选型与接入方法。最后从驱动类封装、用户登录与注册、NFC 辅助设备配网与绑定这三个方面叙述了物联网配套的安卓应用的设计。

4 NFC 配网的智能语音助手设计及关键词识别算法实践

本节将详细介绍智能语音助手嵌入式的软件、硬件的设计框架与流程。另外，虽然本文第二章《基于卷积神经网络的关键词识别系统》对关键词检测模型的理论和训练方法进行了讲解，但模型最终需要部署到的设备为嵌入式设备，真实系统的性能、负载率，甚至内存等都会存在一定限制，因此本节最后将对关键词检测模型进行实物的测试。

4.1 硬件设计

在硬件的电路上，使用嘉立创 EDA 专业版进行原理图与 PCB 设计。本节主要分为“语音助手主板电路设计”、“音频编解码器模块电路设计”、“元件焊接与装配”进行介绍。

4.1.1 语音助手主板电路设计

主板的设计可分为“主控外围”、“电源”、“模块与接口”三部分电路：

（1） 主控外围

主控外围的电路原理图如下图 4-1，主控采用意法半导体 Cortex-M4 架构 STM32F446RET6 系列微控制器，其主要硬件参数如下表 4-1，主频高达 180MHz，具备丰富的外设，同时其具有的浮点运算单元（Floating Point Unit）与 DSP 指令集能加快语音检测的算法浮点、FFT 运算速度。

此部分包含必要的主控以及去耦电容、复位电路、12MHz 谐振的高速外接晶振、32.768kHz 的低速外部晶振（用于 RTC 时钟更精准的定时），蜂鸣器驱动电路、编码器滑钮（用于用户交互输入，同时具备滑动以及按键功能）、以及 TF 卡接口与闪光灯接口（闪光灯另有单独的电路板进行外接）。

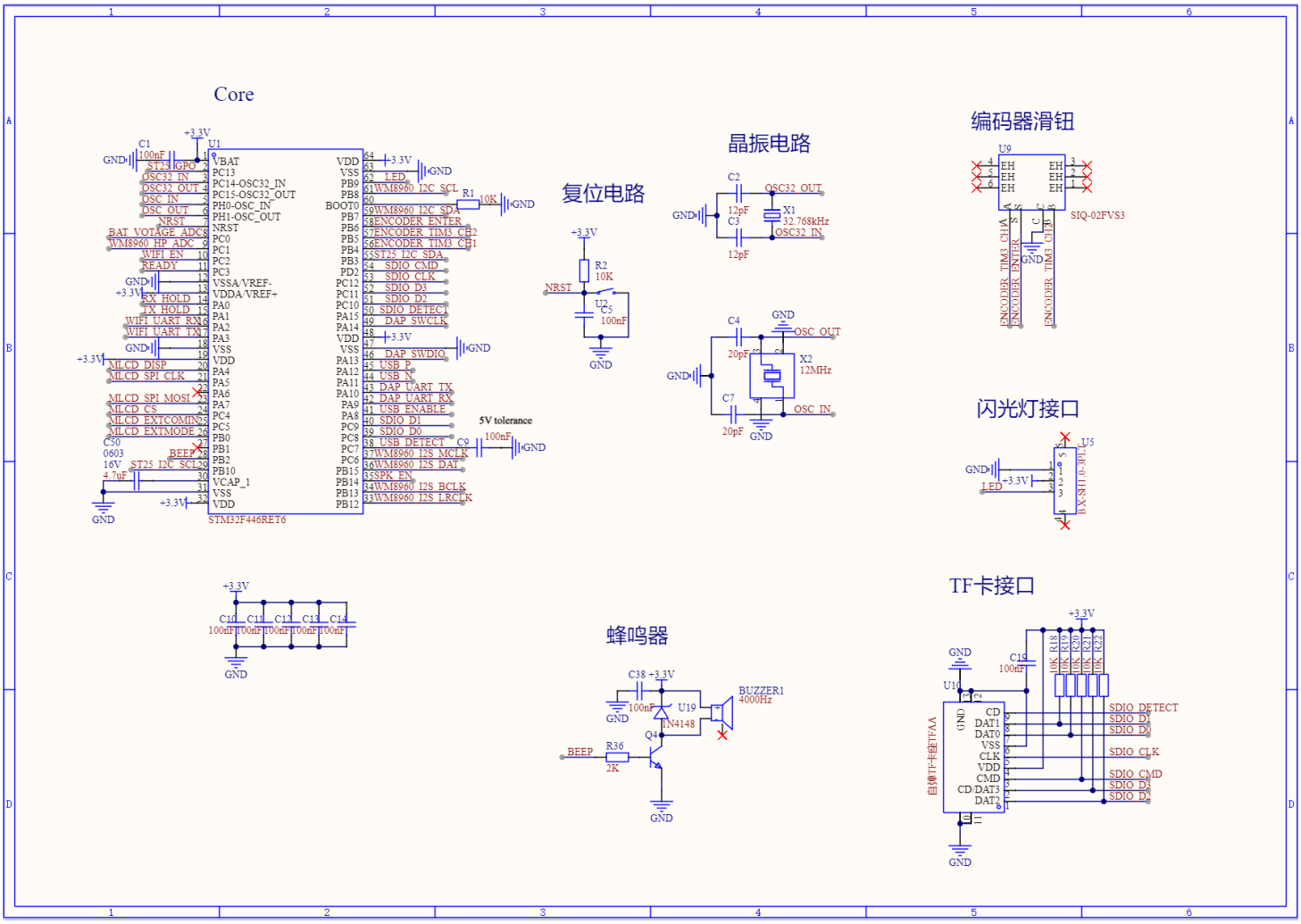


图 4-1　主控外围的电路原理图

表 4-1 STM32F446RET6 硬件参数

	参数
处理器	STM32F446RET6
处理器频率	180MHz
Flash	512KB
RAM	128KB
GPIO	50
I2S	3
SPI	4
UART	6
特性	带 FPU（浮点单元）和 DSP 指令集

（2） 电源

电源部分的电路设计如下图 4-2，USB 接口旁使用三极管控制 DP 引脚是否拉高以控制 USB 设备的连接状态，其提供 5V 的 VBUS 电源，语音助手采用 850mAh 锂电池作为 VBAT 电源，锂电池接口处电阻分压进行 ADC 采样以检测电池电压；为了保证 USB 接入时 VBUS 是唯一的系统电源 VCC 和锂电池充电电源 V_CHARGE，分别采用两个 PMOS 将 VBAT 以及 V_CHARGE(NFC 芯片的能量采集输出的电源)进行电源隔离；“VCC to 5.6V”使用 DC-DC 升压芯片 SX1308，由数据手册[16]可知，改变输出端与 FB 引脚之间的两个电阻阻止可以配置所需升压后的电压，图 4-2 中对应为 R31、R32 电阻，输出的电压 VDD 由公式 4-1 计算得到，其中 V_{REF} 为固定基准电压 0.6V：

$$VDD = V_{REF} \times (1 + \frac{R_{31}}{R_{32}}) = 5.6V \tag{4-1}$$

VDD 供给音频编解码器模块的扬声器 VDD_SPK 引脚的电源，用于驱动两个 1W 8Ω的扬声器；“VCC to 3.3V”使用线性 LDO 稳压芯片 RT9103，稳压输出的 3.3V 作为系统的逻辑电压；锂电池充电芯片采用 TP4056，这是一个单节锂离子电池恒流/恒压线性充电芯片，具有电流反倒流、电流热反馈调节等功能，非常适合便携式设备应用，利用两个指示灯作为充电状态提示，当电池欠压充电时，红灯单独亮起，充电完成时（当充电电流降至设定值的 1/10）绿灯单独亮起。

为了简化外部通信接口，同时满足 USB 转串口 USB 转 TTL、USB 烧录采用沁恒公司的八位微控制器芯片 CH549F，该芯片需要通过 USB 提前烧录好预定的固件程序，故需要用 10KΩ电阻（图 4-2 中对应为 R11 电阻）拉高 DP 脚，烧录完成后，需将该电阻卸下。

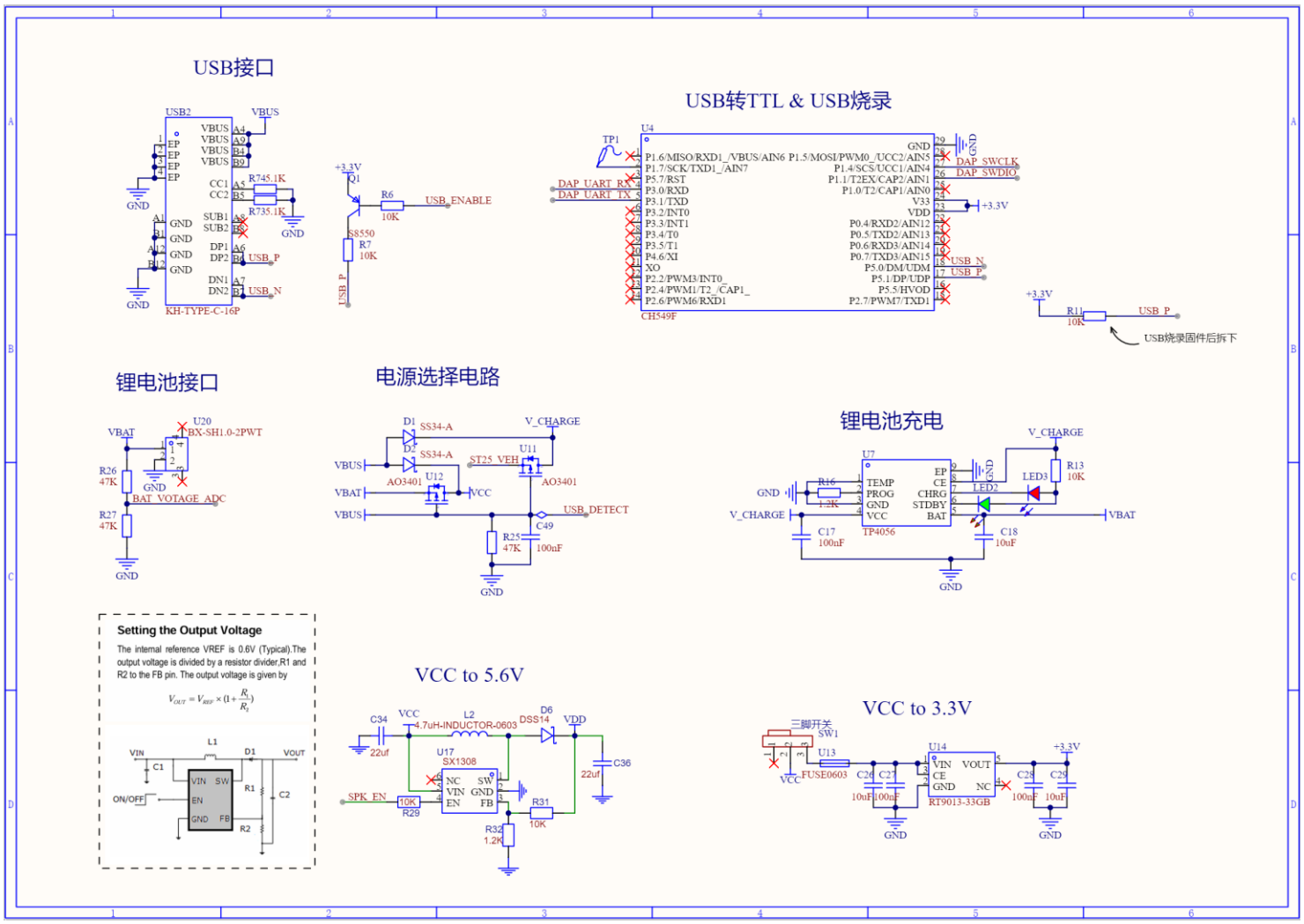


图 4-2　电源部分电路原理图

（3） 模块与接口

设计原理如下图 4-4 所示，无线模块采用乐鑫公司的 ESP32-C3-Mini 物联网模块，这是一款安全稳定、低功耗、低成本的物联网模组，搭载 RISC-V 32 位单核处理器，支持 2.4 GHz Wi-Fi 和 Bluetooth 5 (LE)，使用小龟按键（图中 U24 元件）将 ESP32 的 IO9 引脚拉低，通过串口使用“UART Download Boot”方式对芯片进行程序的烧录，其与主控之间亦用串口进行通信。

NFC 采用意法半导体的 ST25DV16K 芯片，该芯片支持所有 ISO/IEC 15693 调制、编码、副载波模式和速率高达 53 Kbit/s 的自定义快速读取访问，芯片内部自带可供 I2C 或 RF（Radio Frequency Field）写入 16KB 的 EEPROM,还有专用的 256KB 半双工专用缓冲区供 I2C 和 RF 接口之间的数据快速传输，动态 NFC 标签的天线设计基于在应用电路板（PCB）上放置一个线圈。其阻抗与器件内部的调

谐电容值相匹配，以在 13.56 MHz 的频率下形成共振电路。调谐频率的基本公式如式 4-2。

$$f_0 = \frac{1}{2\pi \times \sqrt{L_{antenna} \times C_{tuning}}} \tag{4-2}$$

其中 $L_{antenna}$ 为电路板线圈的电感值，C_{tuning} 为芯片内部谐振电容，根据数据手册[17]可查到芯片的内部谐振电容为 28.5pF，故可算出需要的电感值（式 4-3）：

$$L_{antenna} = \frac{1}{4\pi^2 \times f_0^2 \times C_{tuning}} \approx 4.83\mu H \tag{4-3}$$

使用意法半导体公司在线线圈设计工具[18]可以得到合适的线圈参数，如图 4-3 所示。对于宽 22mm、长 37mm 的线圈大小且线宽 0.254mm、线距 0.254mm 的线圈需要大致绕 10 圈，使得等效电感值为 4.65 μH，与理论计算值 4.83 μH 相近即可。

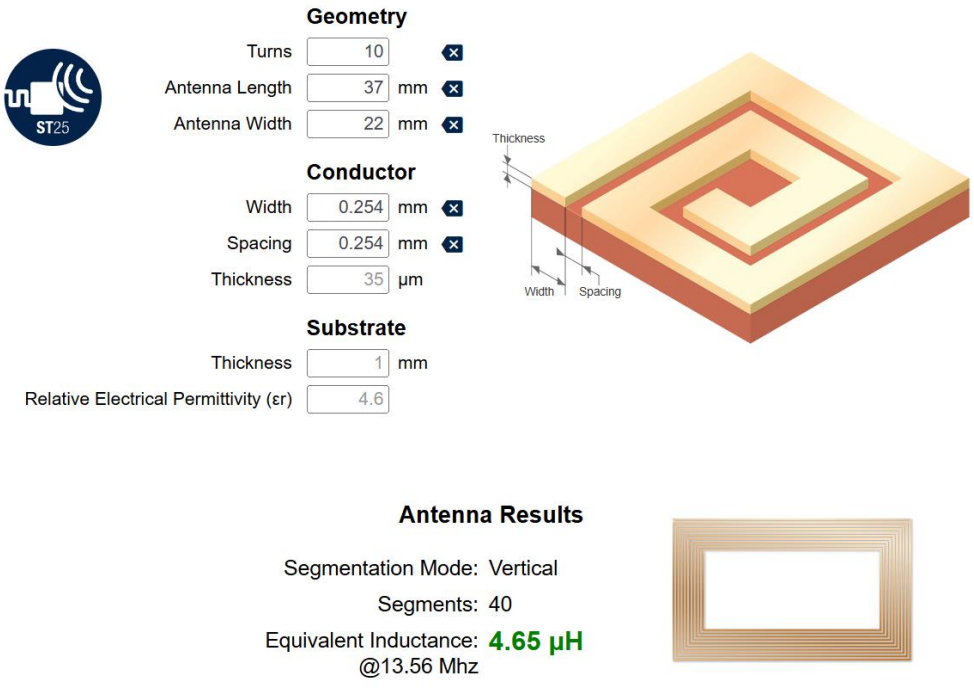


图 4-3　在线设计工具配置 NFC 线圈参数

MLCD（Memory LCD）接口用于 Sharp（夏普）公司的记忆型液晶屏 LS013B7DH06[19]。的 SPI 通信 LS013B7DH06 是一种低功耗反射式显示屏，无背光，在阳光下如同纸面般清晰可读，功耗超低通常在几微瓦（μW）量级，非常适合低功耗应用。

音频模块接口用于外接音频编解码器模块，为了防止数字电源的噪声干扰模拟电路，用磁珠和电阻分别对数字地与模拟地、逻辑电压与模拟电压进行隔离滤波处理。

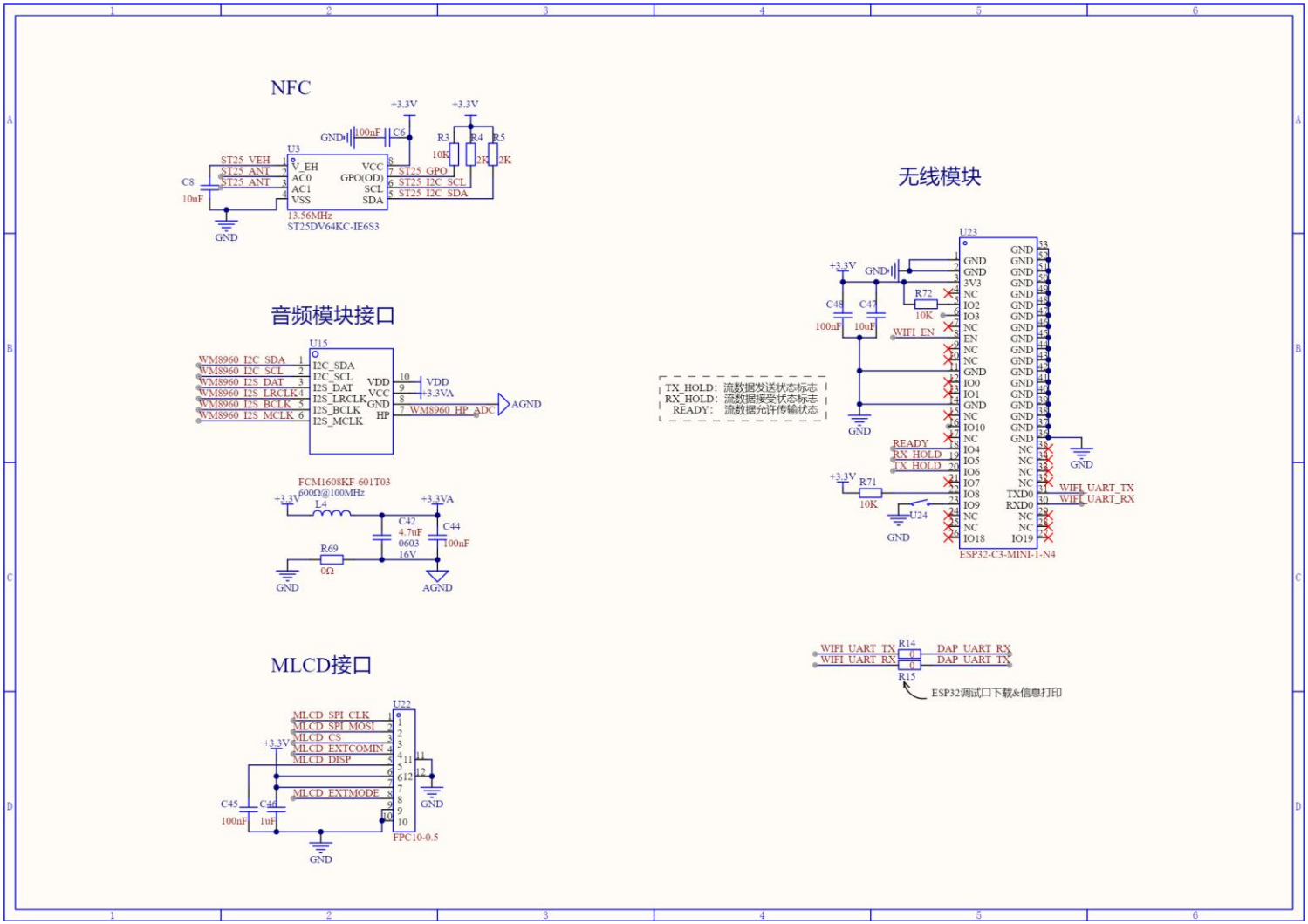


图 4-4　模块与接口部分电路原理图

4.1.2 音频编解码器模块电路设计

模块的原理图如图 4-5 该模块使用 Wolfson Microelectronics 公司的高性能音频编解码器芯片 WM8960G[20]，该芯片集成了高精度 ADC、DAC 单元与模拟前端（耳机/麦克风），具备低噪声麦克风输入、可调增益放大器（PGA），集成立体声耳放，可直接驱动耳机；集成 Class AB 放大器，可直接驱动两个 1W 8Ω的扬声器；且具备可调 EQ、自动音量控制（AVC）、3D 环绕、数字滤波等音频调制功能。故非常适合应用在嵌入式音频系统中。

WM8960G 的内部原理图如图 4-6 所示，STM32 主控通过 I2C 写入 WM8960G 的

相关寄存器参数来对锁相环 PLL 等分频系数、输出输入各环节数字增益器与开关、ADC/DAC 使能、通信协议格式等进行配置。主控与音频模块通过 I2S 互相传输音频信号。为了节省体积，提高高频响应性能，使用硅麦 AOS3729A 作为麦克风。耳机接口使用 4 环的 PJ-393，通过该接口模块可以采集耳机的麦克风信号输入以及检测插入状态或输出音频信号给耳机。

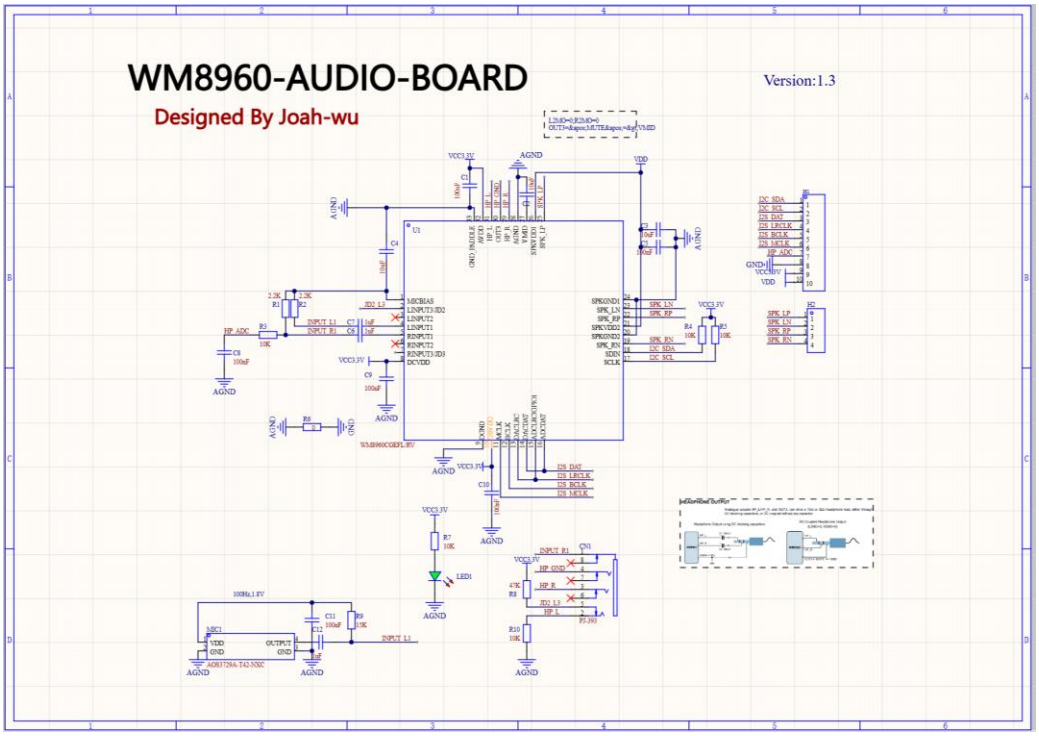


图 4-5　音频编解码器模块电路原理图

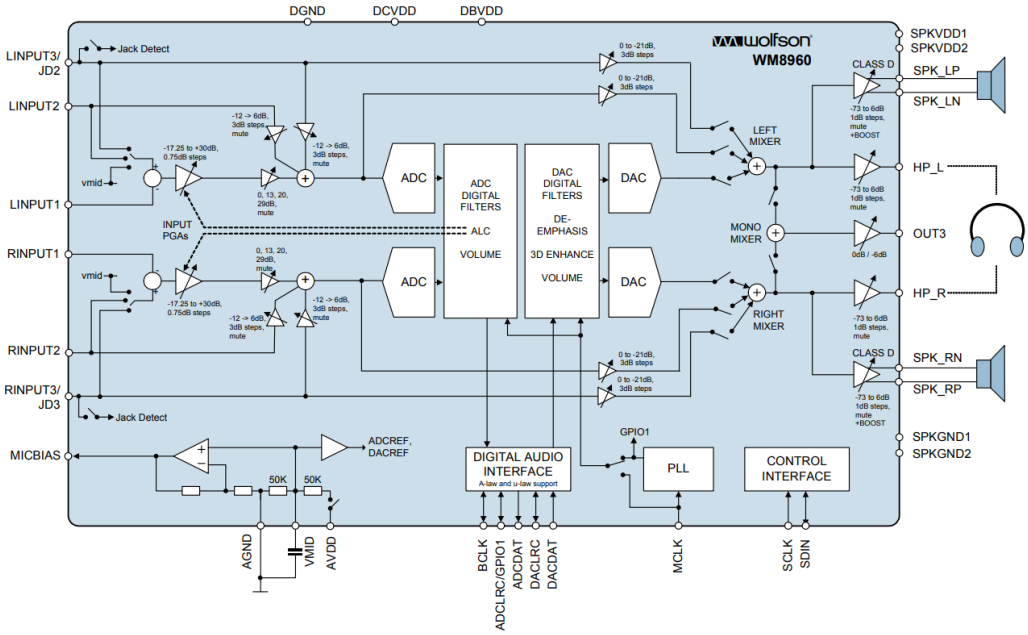


图 4-6　WM8960G 内部原理图

4.1.3 元件焊接与装配

原理图画好后按照电气各项设计规则进行 PCB 设计。语音助手采用四层板，音频编解码模块采用双层板。图 4-7 为语音助手的 PCB 设计图，图 4-8 为音频编解码模块的 PCB 设计图：

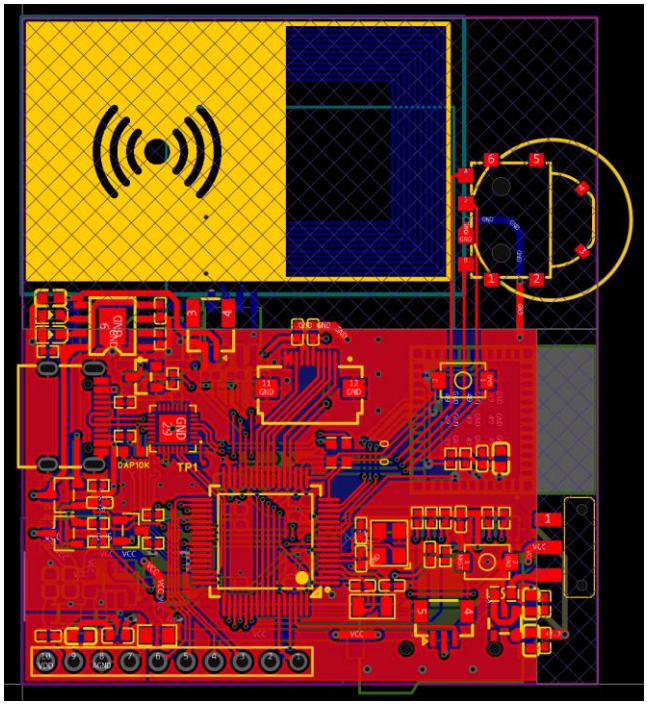


图 4-7　语音助手的 PCB 设计图

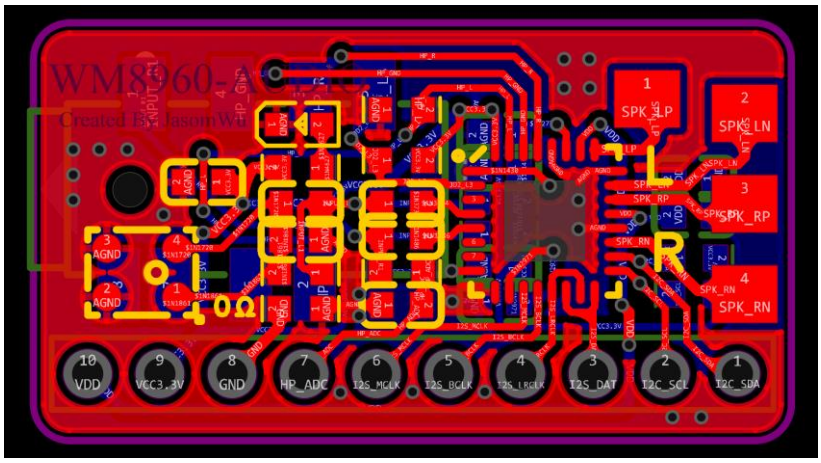


图 4-8　音频编解码模块的 PCB 设计图

使用热风枪与电烙铁进行元器件焊接，最后将两者进行装配，整体实物的三视图如图 4-9 所示：

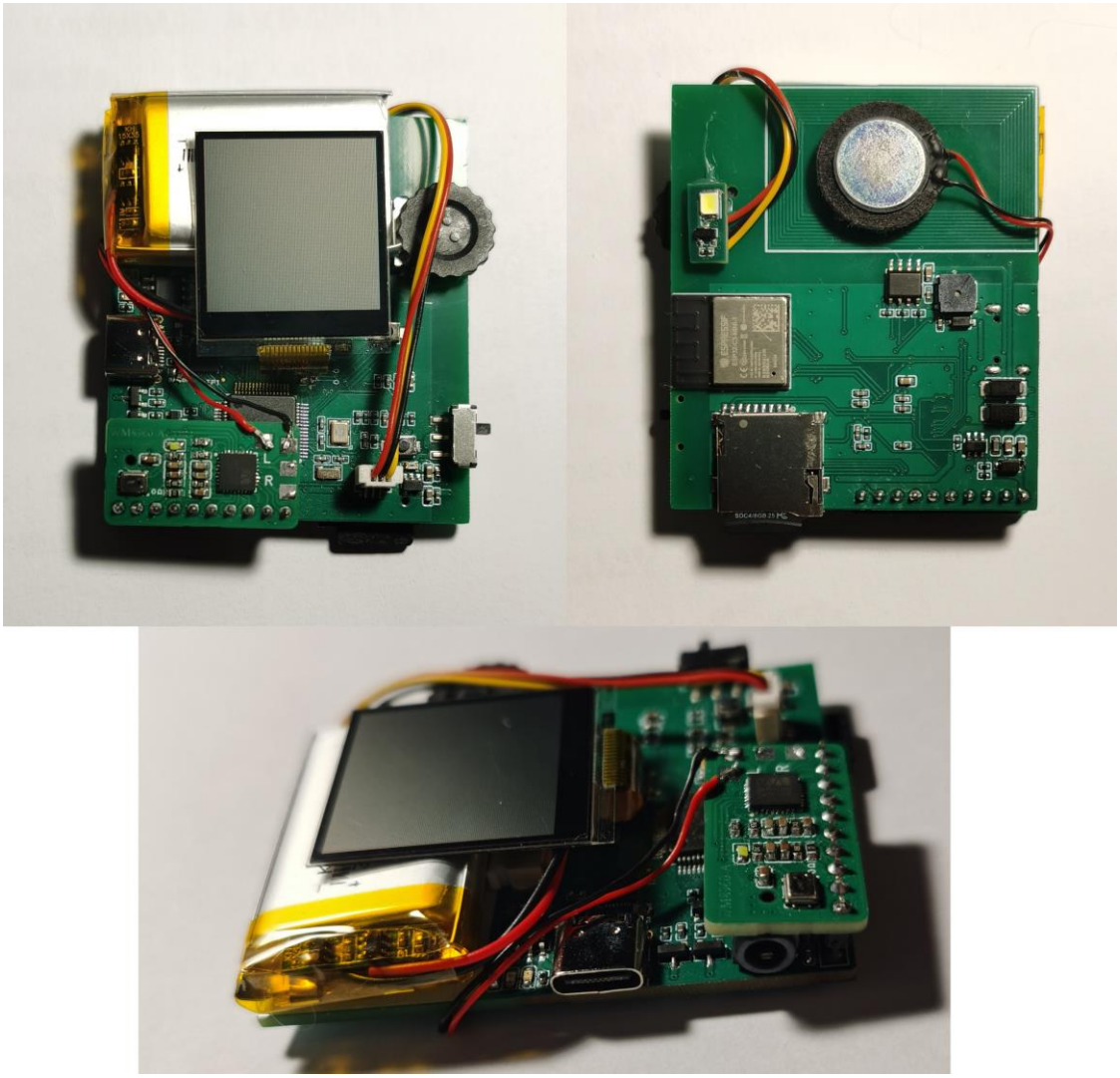


图 4-9　整体实物的三视图

4.2 软件设计

采用 Clion IDE 与交叉编译工具对 STM32 主控和 ESP32 进行开发。本节将按照“软件驱动与开源包配置”、“用户界面系统”、“检测词模型移植验证”逐个进行介绍。

4.2.1 软件驱动配置

嵌入式控制器（MCU） 通过串行协议（SPI/UART 等）完成与外围设备交互，因此在编写传感器驱动前，需要配置好 MCU 的通信接口的驱动程序。为了简化底层驱动的开发流程，这里使用 STM32CubeMX 工具，以图形化操作来配置嵌入式外设，为开发者直接生成底层驱动代码，省去大量移植驱动的实践。需要配置的外

设有 SPI、IIC、IIS、UART、USB、GPIO、TIM、RTC、DMA、ADC、SDIO。如图 4-10 为 STM32CubeMX 工具对 SPI 外设配置的截图。

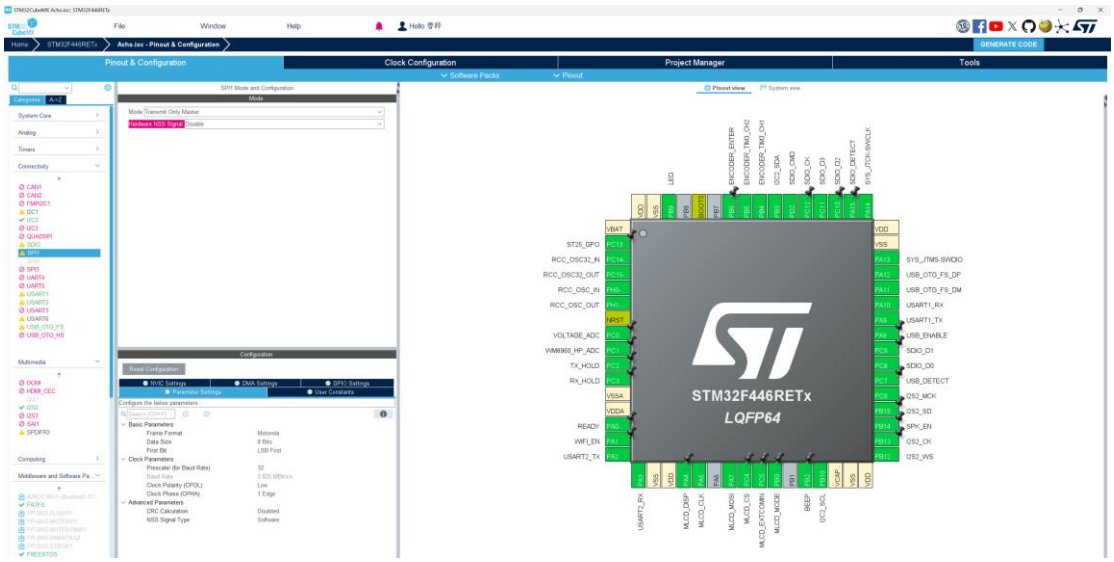


图 4-10　STM32CubeMX 配置 SPI 外设示意图

为实现多任务调度，配置 FreeRTOS（Free Real-Time Operating System）开源的实时操作系统内核库，同时还配置了 FATFS（FAT File System Module）开源轻量级文件系统库用以支持对 FAT12、FAT16、FAT32 和 exFAT 文件系统的访问；以及 st25dv16k 专用的 NFC 库。

4.2.2 用户界面系统

为了更直观地进行人机交互，语音助手采用了开源轻量化图形界面库 LVGL（Light and Versatile Graphics Library）。因其具备内存占用小、跨平台、可自定义样式、支持触摸、动画等特点，适用于各种嵌入式系统。使用编码器滑钮可以用于切换菜单中的光标，短按滑钮可以选择对应条目，双击可以唤出音量调节条，然后通过滑动来调节音量的高低。长按滑钮返回上级目录。如图 4-11 所示，主页菜单从上至下依次为“Play”、“Download”、“Start Chat”、“Toggle Torch”、“Setting”，它们的功能如表 4-2 所示。点击条目可以跳转到新的页面或直接执行某样功能，菜单系统的页面切换关系框图如图 4-12 所示。

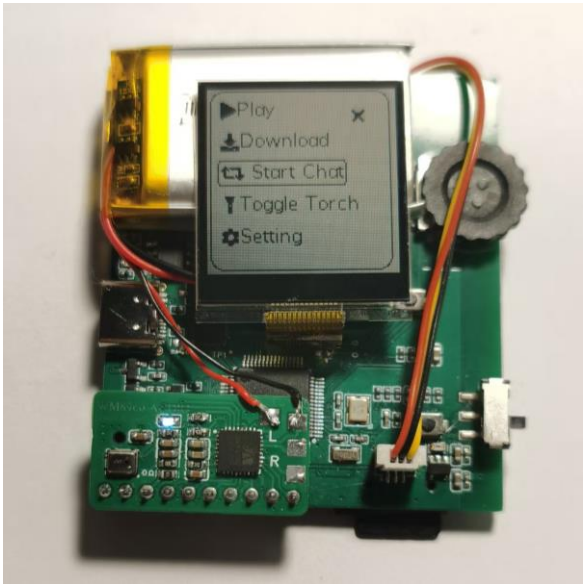


图 4-11　智能语音助手的主页菜单界面

表 4-2　主页菜单各子项功能

	功能
Play	播放音频（MP3 模式）或展示图像
Download	下载指定的音频或图像
Start Chat	开启智能语音助手的唤醒-对话模式
Toggle Torch	打开或关闭闪光灯
Setting	进入设置界面，可选择设备绑定、设备配网、打开 USB

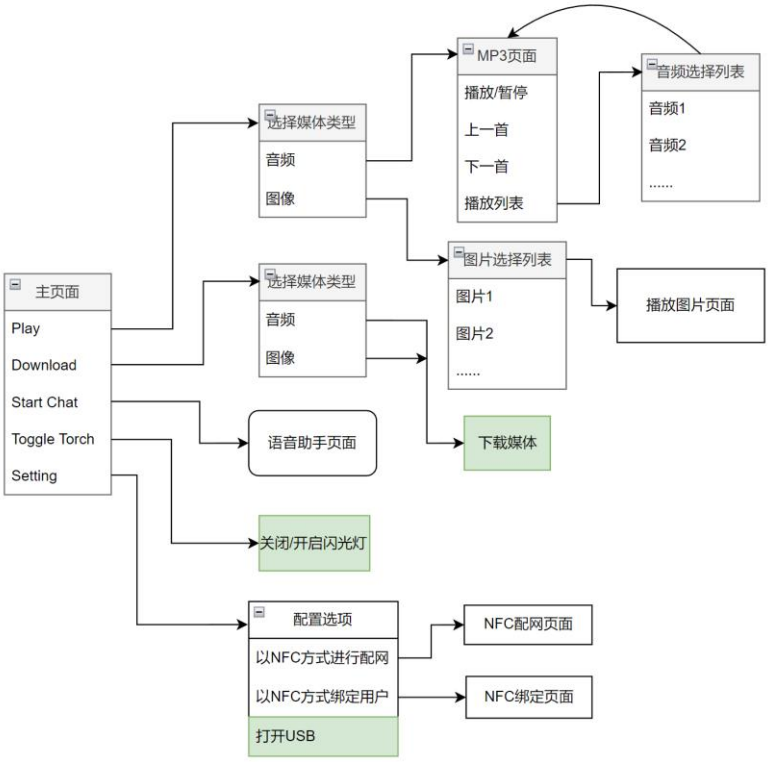


图 4-12　菜单系统的页面切换关系框图

其中对于“Play”与“Download”的后一级“选择媒体类型”页面共用同一套页面（图 4-13a），以及“音频选择列表”与“图片选择列表”也是复用同一套页面（图 4-13b）。在配置选项中，选择以 NFC 方式进行配网或绑定后，系统启动感应模式，然后跳转到对应的提示页面（图 4-14a，图 4-14b）。MP3 页面如图 4-15 所示，三个底部按键功能从左向右依次为“上一曲”、“播放/暂停”、“下一曲”、“播放列表”，当音频被播放时，顶部会显示对应的音频名字，同时“音乐盒子”也会打开示意当前处于播放状态。



（a）　　（b）

图 4-13　共用的页面示意图



（a）



（b）

图 4-14　NFC 绑定与配网界面示意图

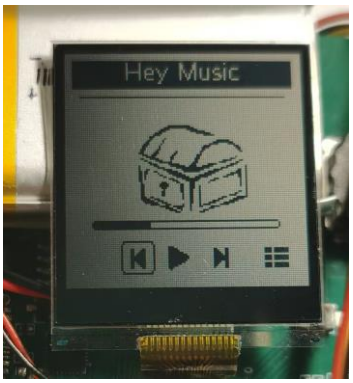
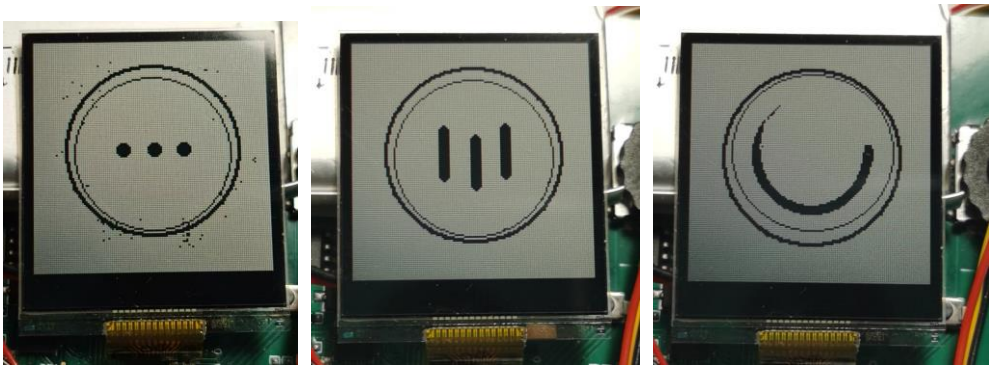


图 4-15　MP3 界面示意图

语音助手界面有四种状态，分别为“待机”、“聆听”、“思考”、“讲话”，其中“待机”为语音助手的非唤醒状态，如图 4-16（a）所示，当识别到关键词时立即转为“聆听”状态，如图 4-16（b）中间的三条竖杠随说话者声音大小上下浮动，表示正等待说话者把话说完。当说话者话语刚落，界面即转变为思考状态，该过程中主要是等待服务器将结果音频回传，如图 4-16（c），等音频接收完成后，切换成“讲话”状态并播放结果音频，该界面与“聆听”状态的界面相同。等音频播放完毕后，界面回到“待机”状态重新开启新一轮的对话循环。



（a）　　（b）　　（c）

图 4-16　语音助手各状态对话界面

4.2.3 检测词模型移植验证

（1）模型移植

ST 公司提供了丰富的软件库与工具，为了便于实现 DSP 功能，ARM 专门创建了 DSP 库 CMSIS-DSP，主要用于数字信号处理。它包括常见的 FFT 操作，并且使用制造商提供的优化 DSP 库代码，可以大大提高计算效率和准确性。其次可以将在计算机上 Pytorch 框架下训练好的模型 onnx 文件输入到 STM32CUBE-AI 工具箱，转换成 C 代码并导入到微控制器中，这样省去了自行用 C 语言写网络推理框架的麻烦。该实验设计的系统结构如图 4-17 所示：

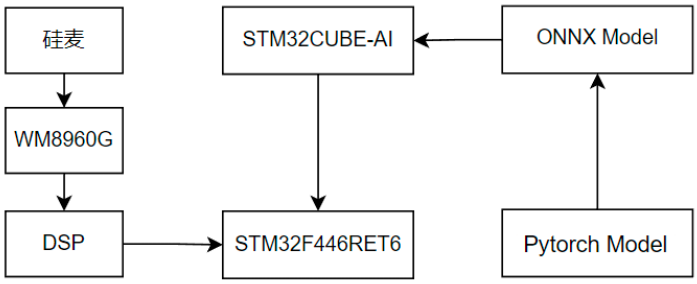


图 4-17　移植实验的系统结构图

在 STM32CubeMX 工具中需要下载“DSP-Library”与“X-CUBE-AI”两个库，如图 4-18 所示。将模型导入到 STM32CUBE-AI 工具箱中进行分析，结果显示运行该模型需要占用 61.29KB 的 Flash 和 10.63KB 的 RAM。STM32F446RET6 完全满足其运行条件，能够在该微控制器上正常运行。分析结果如图 4-19 所示

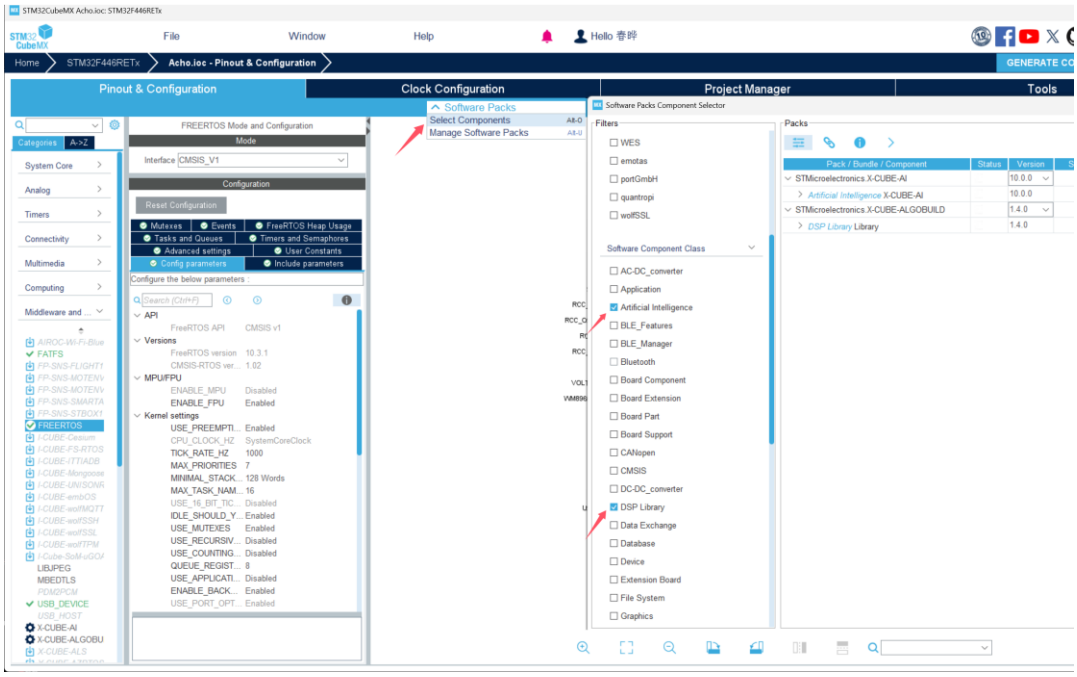


图 4-18　下载 DSP-Libray 与 X-CUBE-AI 库配置示意图

```
Model  file:          wakeword_model.onnx
Total  Flash:        62762  B  (61.29  KiB)
       Weights:        46916  B  (45.82  KiB)
       Library:        15846  B  (15.47  KiB)
Total  Ram:          10888  B  (10.63  KiB)
       Activations:  7476  B  (7.30  KiB)
       Library:        3412  B  (3.33  KiB)
       Input:            5120  B  (5.00  KiB  included  in  Activations)
       Output:           4  B  (included  in  Activations)
```

图 4-19　硬件资源占用分析结果

（2）模型验证

STM32F446 通过音频编码器模块实时收集硅麦输入的音频数据，为了提高唤醒词检测的效率，即尽可能捕获到唤醒词的音频片段，采用音频波形的过零率[21]来满足这一点。过零率是指信号中相邻两个采样点符号不同的次数（即信号从正变负或从负变正）与总采样点数之比。设一帧信号为 $x[n]$，帧长为 N，则过零率可用公式 4-4 来计算：

$$ZCR = \frac{1}{2N}\sum_{n=1}^{N-1}\left|\mathrm{sgn}(x[n]) - \mathrm{sgn}(x[n-1])\right| \tag{4-4}$$

其中：

$$\mathrm{sgn}(x) = \begin{cases} 1, & x \geq 0 \\ -1, & x < 0 \end{cases}$$

一般而言，人声的浊音部分的 ZCR 普遍低于环境噪音的 ZCR，人声的清音部分的 ZCR 会高于环境噪音的 ZCR。图 4-20 为唤醒词“Hey Acho”的原始音频波形图与过零率对比图，图 4-21 为安静卧室中背景噪声的原始音频波形图与过零率对比图。“Hey”开头的‘h’是一个轻轻呼气的声音，没有声带振动，所以属于清辅音，故可以设定合适的过零率阈值，检测大于阈值的部分作为人声的触发起点。

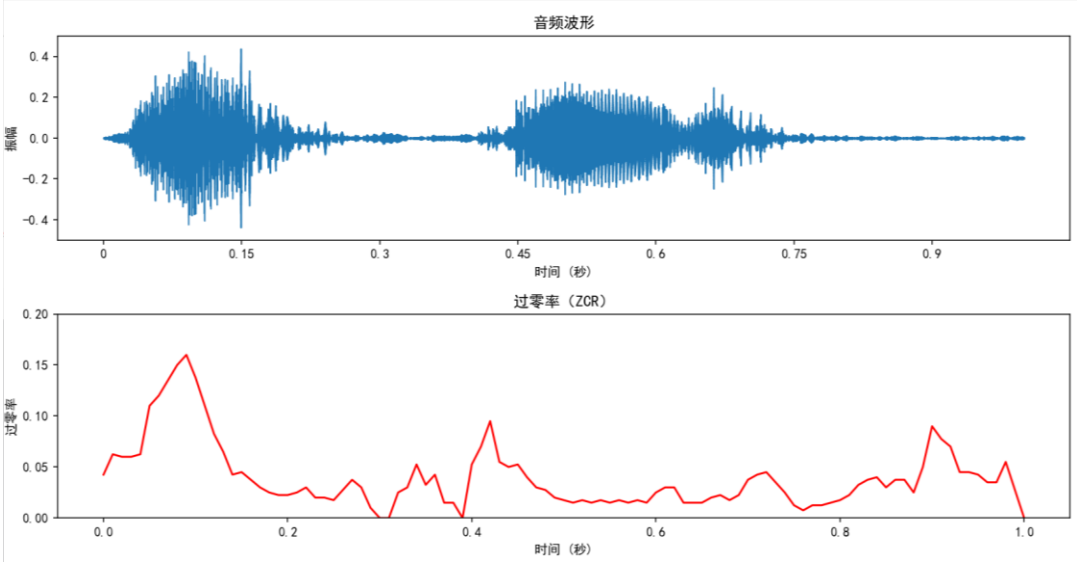


图 4-20　唤醒词“Hey Acho”的原始音频波形图与过零率对比图

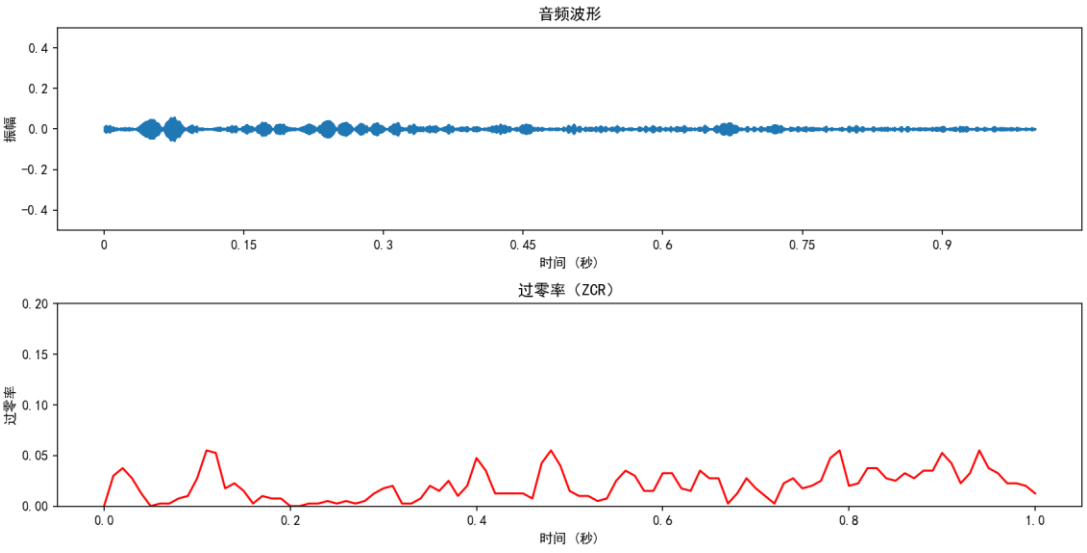


图 4-21　安静卧室中背景噪声的原始音频波形图与过零率对比图

其次，低声量且更高频的噪声通常拥有与清音相近的 ZCR 大小，为了避免误判，可以结合短时能量特性来辅助检测。语音信号的能量是反映音频强弱（音量

大小）的一个基本特征，对于每一帧的短时能量采用平均能量公式 4-5 来计算：

$$E_{mean} = \frac{1}{N}\sum_{n=0}^{N-1} x[n]^2 \tag{4-5}$$

在语音助手“待机”界面（图 4-12a）显示之前，都会对环境的噪声的过零率与平均能量进行提前采样，然后使用公式 4-6、公式 4-7 分别计算过零率动态阈值与平均能量动态阈值：

$$TH_{zcr} = BGZCR_{mean} + \alpha \times BGZCR_{std} \tag{4-6}$$

$$TH_{energy} = BGE_{mean} + \beta \times BGE_{std} \tag{4-7}$$

其中，$BGZCR_{mean}$、BGE_{mean} 分别为环境噪声的平均过零率与平均能量，α、β 为经验参数，通常取 1~2，$BGZCR_{std}$、BGE_{std} 分别为环境噪声的过零率标准差与能量标准差，即：

$$BGZCR_{std} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1}(BGZCR[n] - BGZCR_{mean})^2}$$

$$BGE_{std} = \sqrt{\frac{1}{N}\sum_{n=0}^{N-1}(BGE[n] - BGE_{mean})^2}$$

计算完阈值后，启动关键词监听环节，采用滑动采样的方式实时取若干个连续的帧，计算它们的平均过零率以及平均能量，当平均过零率与平均能量同时大于各自的动态阈值时则判为人声片段，触发唤醒词采样过程，否则，视为非人声片段。这个组合判断的伪代码如图 4-22 所示：

```
if energy > threshold_energy and zcr > threshold_zcr:
    is_voice = True
else:
    is_voice = False
```

图 4-22　组合判断伪代码

接着控制器调用 DSP 库从收集到的人声片段中提取 MFCC 特征，以获得声音特征向量。微控制器内部通过模型计算声音特征向量，并据此进行检测词识别。设唤醒词的概率阈值为 0.5，当模型推理出来的概率大于 0.5，则判断为“唤醒词”，语音助手界面即跳转至“聆听”状态界面（图 4-16b），即完成检测词系统的所有推理工作。

（3）识别准确率检验

从 PC 端上训练好的模型移植到嵌入式平台上，模型的推理准确率会因为压缩模型时浮点转定点、DSP 库算法计算精度不同等问题导致嵌入式平台上检测的准确率与 PC 上的准确率差之甚远。故需要将 PC 上的同一份测试集数据加载到嵌入式模型上进行准确率检验。

首先，确保硬件上 SD 卡已正确连接到 STM32，并在 CubeMX 中配置 FATFS 和 SD 卡驱动。软件上需要实现 FATFS 的文件遍历功能，读取 WAV 格式的测试音频文件，按文件夹区分唤醒词和非唤醒词，输入到模型中计算概率并统计准确率。

在 **main.c** 中初始化 SD 卡和 FATFS，挂载文件系统后，逐个读取文件内容到缓冲区。对于每个音频文件，调用模型推理函数（ai_asrmodel_run）得到输出概率值，若概率大于 0.5 则判定为唤醒词，否则为非唤醒词，同时根据文件所在文件夹确定真实标签，比较预测结果与真实标签，累计正确预测次数。最后，计算总准确率并通过 UART 输出结果。

测试集一共有 60 个唤醒词数据与 70 个非唤醒词，将测试集分别在 PC 与嵌入式设备上进行测试，得到他们的混淆矩阵如图 4-23 与图 4-24 所示。可以计算出在 PC、嵌入式设备上推理模型准确率如表 4-3 所示，可见在 STM32F446 上运行的模型的推理准确率稍低于 Pytorch 平台上推理的准确率，准确率仅下降了不到 4%。准确率的损失来源于将模型转换为可在微控制器上运行的代码的过程，这一转换是通过 STM32CUBE-AI 工具箱完成的。在此过程中，工具箱对模型进行了压缩，以牺牲部分准确率为代价，换取更快的运行速度和更小的模型体积。

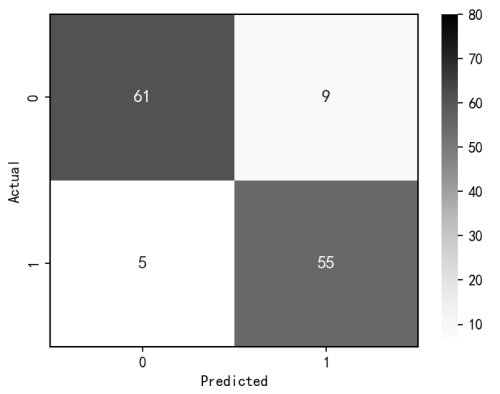


图 4-23　PC 模型推理的混淆矩阵

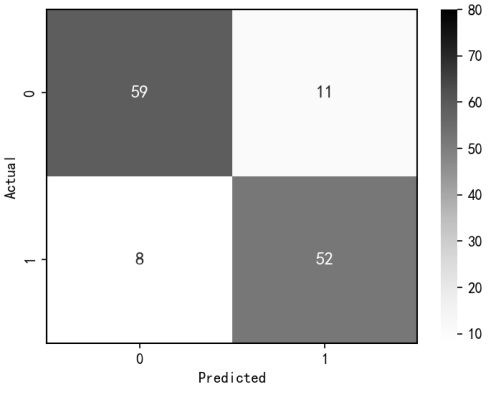


图 4-24　嵌入式设备模型推理的混淆矩阵

表 4-3 不同平台测试模型的准确率

模型	Pytorch 运行的模型	STM32F446 运行的模型
测试准确率	89.23%	85.38%

4.3 本章小结

本章介绍了智能语音助手终端的软硬件设计方法与流程。对语音助手主板“主控”、“电源”、“模块与接口”部分以及音频编解码模块的电路设计进行说明。软件部分主要使用 STM32CubeMX 进行驱动代码以及开源包的生成，接着介绍智能语音助手的菜单系统与各 ui 的设计。使用 X-CUBU-AI 工具将 Pytorch 训练的检测词模型移植至嵌入式设备上，提出了过零点率、平均能量联合评判的方法来捕捉人声信号以触发关键词识别过程，最后使用测试集在嵌入式设备上对模型进行了推理准确率检验，与 Pytorch 框架下运行模型的推理准确率相比较，STM32F446 模型推理准确率仅稍低于 Pytorch 模型推理准确率不到 4%。

5 结论与展望

5.1 课题结论

本研究成功实现了基于 NFC 配网的智能语音助手与私人物联网平台，通过软硬件的协同设计，解决了隐私安全、设备互联和语音交互等关键问题。在私人物联网平台方面，基于 Django 框架构建的系统采用三令牌动态认证机制，有效保障了数据传输安全；智能语音助手硬件采用 STM32F446RET6 微控制器为核心，集成了 NFC、Wi-Fi 和音频编解码功能模块，实现了紧凑的硬件设计；提出的轻量级 CNN-MFCC 关键词检测模型在嵌入式设备上达到 85.38%的识别准确率；创新的 NFC 配网方案简化了设备绑定流程，提升了用户体验。这些成果为智能家居设备的隐私保护和便捷交互提供了可行的技术方案。

5.2 未来展望

展望未来，本研究可在以下方面进一步深化：在技术层面，可探索更高效的神经网络架构和自适应噪声抑制算法来提升语音识别的实时性和鲁棒性;在系统功能方面，需要加强边缘计算与云平台的协同优化，并提升跨平台兼容性；NFC 技术应用可向无源设备供电和动态安全认证等方向拓展。此外，商业化推广中的成本控制和标准化建设也值得关注，包括硬件模块化设计以降低 BOM 成本，以及参与制定行业安全标准等。这些后续工作将有助于推动本研究成果的实际应用和产业化落地。

参考文献

[1] Coskun, V., Ok, K., & Ozdenizci, B. (2013). Near Field Communication (NFC): From Theory to Practice. Wiley.

[2] Django Software Foundation. (2023). Django Documentation. (n.d.) [2025-04-25] https://docs.djangoproject.com

[3] Sakoe, H., & Chiba, S. (1978). "Dynamic Programming Algorithm Optimization for Spoken Word Recognition." IEEE Transactions on Acoustics, Speech, and Signal Processing, 26(1), 43-49.

[4] Rabiner, L. R. (1989). "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition."Proceedings of the IEEE, 77(2), 257-286.

[5] Bilmes, J. A. (1998). "A Gentle Tutorial of the EM Algorithm and Its Application to Parameter Estimation for Gaussian Mixture and Hidden Markov Models."Technical Report, UC Berkeley.

[6] Cover, T., & Hart, P. (1967). "Nearest Neighbor Pattern Classification." IEEE Transactions on Information Theory, 13(1), 21-27.

[7] Aha, D. W., Kibler, D., & Albert, M. K. (1991). "Instance-Based Learning Algorithms."Machine Learning, 6(1), 37-66.

[8] Sainath, T. N., et al. (2015). "Convolutional Neural Networks for Small-Footprint Keyword Spotting."Interspeech, 1478-1482.

[9] Howard, A. G., et al. (2017). "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications."

[10] Graves, A., et al. (2013). "Speech Recognition with Deep Recurrent Neural Networks." ICASSP, 6645-6649.

[11] Vaswani, A., et al. (2017). "Attention Is All You Need."NeurIPS, 5998-6008.

[12] Zhang, Y., et al. (2017). "Hello Edge: Keyword Spotting on Microcontrollers." arXiv:1711.07128.2

[13] Stone, M. (1974). "Cross-Validatory Choice and Assessment of Statistical Predictions."Journal of the Royal Statistical Society, 36(2), 111-147.

[14] Davis, S. B., & Mermelstein, P. (1980). "Comparison of Parametric Representations for Monosyllabic Word Recognition."

[15] Vanhoef, M., & Piessens, F. (2017). "Key Reinstallation Attacks: Forcing Nonce Reuse in WPA2." ACM CCS, 1313–1328.

[16] Semtech Corporation. SX1308 2A DC-DC Step-Up Converter Datasheet [EB/OL].(n.d.)[2025-4-20].https://www.alldatasheet.com/datasheet-pdf/pdf/1131831/SEMTECH/SX1308.html

[17] STMicroelectronics. ST25DV16K Dual-Interface EEPROM Datasheet [EB/OL].(n.d.)[2025-05-05].https://www.alldatasheet.com/datasheet-pdf/pdf/1549712/STMICROELECTRONICS/ST25DV16K.html

[18] STMicroelectronics. ST25 Antenna Matching Tool [EB/OL]. (n.d.) [2025-05-05]. https://eds.st.com/antenna/

[19] Sharp Corporation. LS013B7DH06 Memory LCD Datasheet [EB/OL]. (n.d.)[2025-05-06].https://www.alldatasheet.com/datasheet-pdf/pdf/1253152/SHARP/LS013B7DH06.html

[20] https://www.alldatasheet.com/datasheet-pdf/view/1244316/CIRRUS/WM8960.html

[21] Rabiner, L. R., & Schafer, R. W. (1978). Digital Processing of Speech Signals.Prentice Hall.